

Flutter từ cơ bản đến nâng cao — Tập 2: Trung cấp

State (ChangeNotifier + provider), Future/Stream, HTTP + JSON, SQLite, animation, TTS + thông báo, testing

Flutter 3.47.5 · Dart 3.13.4 · provider 6.1.5+1 · go_router 18.0.2 · sqflite 2.4.4

Bản dựng: 2026-09-30 — Markdown là nguồn chính (books/flutter/vol2-trung-cap/)

Tham khảo docs.flutter.dev (CC BY 3.0) và dart.dev (CC BY 4.0) — xem Nguồn tham khảo cuối mỗi chương

Kiểm tra dấu tiếng Việt: ă ằ ẳ ẵ ơ ư ơ đ ư đ — Đường ướn, trắng khuyết, Ưu tiên

Mục lục

Chương 1 — Quản lý state: setState → ChangeNotifier → provider (MVVM)

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 2 — Bất đồng bộ: Future, async/await, Stream, isolate

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 3 — HTTP và JSON

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 4 — Lưu trữ offline: shared_preferences và SQLite (sqflite)

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 5 — Animation: implicit, explicit, Hero

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 6 — Device APIs: đọc to (text-to-speech) và thông báo cục bộ

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 7 — Testing: unit, widget, integration; fake và mock

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 8 — App mẫu Tập 2: "Số Từ Vựng" (bản tập dượt cho app TOEIC)

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

GLOSSARY — Thuật ngữ Flutter (Anh → Việt)

Chương 1 — Quản lý state: setState → ChangeNotifier → provider (MVVM)

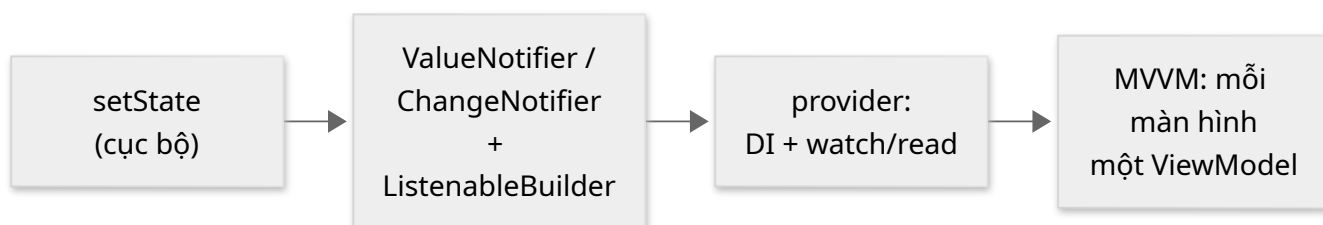
Mục tiêu

- Phân biệt **state cục bộ** (ephemeral) và **state của app** theo docs chính thức.
- Dùng `ChangeNotifier` + `ListenableBuilder` (cách Learning Pathway dạy, không cần package).
- Dùng package **provider** để tiêm phụ thuộc (DI) và lắng nghe: `ChangeNotifierProvider`, `context.read`, `context.watch`, `context.select`, `Consumer`.
- Hiểu mô hình **MVVM** (View + ViewModel) mà trang Architecture recommendations khuyến.
- Biết so sánh với Riverpod, Bloc, signals — và vì sao sách chọn provider.

Giải thích đơn giản

Docs ("Ephemeral vs app state"): state **cục bộ** là thứ chỉ một widget cần (tab đang chọn, ô đang gõ) → `setState` là đủ. State **của app** là thứ nhiều màn hình cần hoặc phải giữ lâu (danh sách từ vựng, cài đặt, đăng nhập) → đưa ra khỏi widget, vào một class riêng.

Con đường docs chính thức gợi ý, từ đơn giản tới đầy đủ:



Angular	Flutter (sách dùng)
Service <code>@Injectable</code> có state (signal/BehaviorSubject)	<code>class CartModel extends ChangeNotifier</code>
<code>this.items.set([...])</code> / <code>subject.next()</code>	sửa field private + <code>notifyListeners()</code>
<code>providers: [CartService]</code> ở component/route	<code>ChangeNotifierProvider(create: (_) => CartModel(), child: ...)</code>
<code>inject(CartService)</code> rồi gọi hàm	<code>context.read<CartModel>().add(item)</code>
Đọc signal trong template (tự cập nhật)	<code>context.watch<CartModel>()</code> hoặc <code>Consumer<CartModel></code>
<code>computed(() => cart.count())</code> chỉ đổi khi count đổi	<code>context.select<CartModel, int>((c) => c.count)</code>
<code>ngOnDestroy</code> của service theo component	provider tự gọi <code>dispose()</code> khi widget bị gỡ

Ví dụ

ChangeNotifier — "service có state"

examples/lib/chapters/ch01/cart_model.dart (theo đúng ví dụ giỏ hàng trong docs "Simple app state management"):

```
class CartModel extends ChangeNotifier {
  final List<CatalogItem> _items = [];

  List<CatalogItem> get items => List.unmodifiable(_items);
  int get count => _items.length;
  int get totalPrice => _items.fold(0, (sum, i) => sum + i.price);

  void add(CatalogItem item) {
    _items.add(item);
    notifyListeners(); // ≈ signal.set() / subject.next()
  }

  void removeAll() {
    _items.clear();
    notifyListeners();
  }
}
```

Không cần package: ListenableBuilder (Learning Pathway)

```
class CounterView extends StatelessWidget {
  const CounterView({super.key, required this.viewModel});

  final CounterViewModel viewModel;

  @override
  Widget build(BuildContext context) {
    return ListenableBuilder(
      listenable: viewModel,
      builder: (context, _) => Row(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          Text('Đếm: ${viewModel.value}'),
          IconButton(tooltip: 'Tăng', onPressed: viewModel.increment, icon: const
Icon(Icons.add)),
        ],
      ),
    );
  }
}
```

ViewModel được truyền qua constructor. Tập 1 (app "Việc Cần Làm") làm đúng như vậy.

Có provider: DI + lắng nghe

examples/lib/chapters/ch01/provider_demo.dart:

```
ChangeNotifierProvider(
  create: (_) => CartModel(), // provider tự dispose CartModel khi widget bị gỡ
  child: const Column(mainAxisSize: MainAxisSize.min, children: [CatalogList(),
    Divider(), CartSummary()]),
);

// Trong CatalogList – chỉ GỌI HÀM, không cần build lại khi giỏ đổi:
onPressed: () => context.read<CartModel>().add(item),

// Trong CartSummary – HIỂN THỊ, cần build lại:
Consumer<CartModel>(
  builder: (context, cart, child) => ListTile(
    title: Text('Giỏ: ${cart.count} món – ${cart.totalPrice}k'),
    trailing: TextButton(onPressed: cart.count == 0 ? null : cart.removeAll, child:
const Text('Xóa hết')),
  ),
);
```

Quy tắc nhớ nhanh:

- `context.watch<T>()` — trong `build`, muốn **build lại** khi `T` đổi.
- `context.read<T>()` — trong **callback** (`onPressed...`), chỉ lấy đối tượng. Không gọi `read` trong `build` để hiển thị.
- `context.select<T, R>(fn)` — chỉ build lại khi **kết quả** `fn` đổi.
- `Consumer<T>` — như `watch` nhưng chỉ phần `builder` build lại (dùng sâu trong cây).

MVVM trong app mẫu "Sổ Từ Vựng"

App Tập 2 theo khuyến nghị kiến trúc chính thức: **mỗi màn hình một ViewModel**, dữ liệu lấy qua **repository** được tiêm bằng provider. `examples/lib/config/router.dart`:

```
GoRoute(
  path: '/words',
  builder: (context, state) => ChangeNotifierProvider(
    create: (context) => WordListViewModel(repository: context.read()),
    child: const WordListScreen(),
  ),
),
```

`create` chạy **một lần** cho màn hình; khi rời màn hình, provider tự `dispose()` ViewModel. Ở gốc app, `MultiProvider` cung cấp repository và service (`config/dependencies.dart`, Chương 8). Trong View:

```
final vm = context.watch<WordListViewModel>(); // build lại khi vm.notifyListeners()
// ...
onChanged: (text) => _debouncer(() => context.read<WordListViewModel>().search(text)),
```

Và `app.dart` chỉ build lại `MaterialApp` khi **theme** đổi:

```
final themeMode = context.select<SettingsViewModel, ThemeMode>((vm) => vm.themeMode);
```

Kết quả test thật

`flutter test --reporter expanded test/chapters/ch01_test.dart` (2026-09-30):

```
00:00 +0: Chương 1 – quản lý state CartModel báo listener khi thêm / xóa
00:00 +1: Chương 1 – quản lý state CounterView (ChangeNotifier + ListenableBuilder)
00:01 +2: Chương 1 – quản lý state ProviderCartDemo: read để gọi hàm, Consumer để hiển thị
00:01 +3: Chương 1 – quản lý state Bài 1: context.select chỉ build lại khi số lượng đổi
00:01 +4: Chương 1 – quản lý state Bài 2: CartNotifier tạo list mới mỗi lần đổi
00:01 +5: All tests passed!
```

Đi sâu

Các lựa chọn khác (so sánh)

Trang "State management options" liệt kê cách có sẵn (`setState`, `ValueNotifier`/`InheritedNotifier`, `InheritedWidget`) và dẫn tới các package cộng đồng. Số liệu kiểm tra 2026-09-30 (chi tiết ở [STACK.md](#)):

Package (phiên bản mới nhất)	Ý tưởng	Giống Angular	Ghi chú
<code>provider 6.1.5+1</code>	DI + lắng nghe <code>ChangeNotifier</code>	DI + service	Docs chính thức khuyên dùng cho DI ; sách chọn
<code>flutter_riverpod 3.4.3</code>	Provider an toàn compile-time, cache async, không cần <code>BuildContext</code>	DI + RxJS store	Mạnh; nhiều khái niệm hơn; <code>pub.dev</code> không gắn tag web cho bản 3.4.3
<code>flutter_bloc 9.1.1</code>	Sự kiện → Bloc → State (Stream)	NgRx	Nhiều code mẫu (boilerplate), rất tường minh
<code>signals 7.1.0</code>	<code>signal</code> , <code>computed</code> , <code>effect</code> có auto-tracking	Angular Signals	Quen thuộc nhất với Nobin; cộng đồng nhỏ hơn

Quyết định của sách (và của app TOEIC — Task 9): `ChangeNotifier` + `provider`. Lý do: (1) docs chính thức khuyên (MVVM + `ChangeNotifier` + `provider`); (2) `ChangeNotifier` có sẵn trong SDK; (3) map thẳng sang service + DI của Angular; (4) ít khái niệm → ít lỗi khi một agent khác làm theo.

ViewModel không được biết về widget

ViewModel chỉ chứa **state + hàm**; không import `BuildContext`, không gọi `Navigator`, không hiện `SnackBar`. Muốn báo một việc "một lần" (ví dụ thông báo), đặt field `message` rồi View hiện và gọi `consumeMessage()` — xem `SettingsViewModel`. Nhờ vậy test ViewModel bằng `test()` thường, không cần widget (Chương 7).

`notifyListeners()` và hiệu năng

`notifyListeners()` báo **mọi** người nghe. Nếu nhiều widget `watch` một ViewModel lớn, dùng `select` hoặc `Consumer` đặt sâu trong cây để giảm phần build lại. `ValueNotifier` chỉ báo khi `value` **khác** (`==`) giá trị cũ.

Lỗi và bẫy thường gặp

- `context.read` **trong build để hiển thị** → UI không cập nhật. Dùng `watch/select/Consumer`.
- `context.watch` **trong callback** → lỗi assert của provider. Dùng `read`.
- `ProviderNotFoundException`: widget nằm **ngoài** (phía trên) provider, hoặc sai kiểu generic (`Provider<SqliteWordRepository>` nhưng đọc `WordRepository`). Khai báo provider theo **interface**.
- **Tạo ViewModel trong build** (`WordListViewModel(...)` mỗi lần build) → mất state, rò rỉ. Dùng `create:` của provider.
- **Sửa list rồi gán lại cùng đối tượng cho `ValueNotifier`** → không báo (vì `==` cùng tham chiếu). Tạo list mới (Bài 2).
- **Quên `notifyListeners()` sau khi đổi field.**

Tóm tắt

- State cục bộ → `setState`. State app → `ChangeNotifier` (ViewModel/service).
- Không package: `ListenableBuilder`. Có provider: `ChangeNotifierProvider` + `read/watch/select/Consumer`.
- MVVM theo docs: mỗi màn hình một ViewModel tạo bằng `create:`; repository tiêm qua `MultiProvider`.
- Riverpod/Bloc/signals đều tốt; sách và app TOEIC dùng provider vì docs khuyên và đơn giản.

Bài tập (có lời giải)

Bài 1. Viết `CartCountBadge` hiển thị số món trong giỏ, **chỉ build lại khi số lượng đổi** (không build lại khi `notifyListeners()` được gọi mà số lượng giữ nguyên). Viết test đếm số lần build.

▼ Lời giải

examples/lib/chapters/ch01/exercise_solution.dart:

```
class CartCountBadge extends StatelessWidget {
  const CartCountBadge({super.key});

  static int buildCount = 0; // chỉ để test đếm số lần build

  @override
  Widget build(BuildContext context) {
    buildCount++;
    final count = context.select<CartModel, int>((cart) => cart.count);
    return Badge(label: Text('$count'), child: const Icon(Icons.shopping_cart));
  }
}
```

Test: build lần 1 → buildCount = 1; add → 2; removeAll → 3; gọi removeAll lần nữa (vẫn gọi notifyListeners() nhưng count vẫn 0) → **vẫn 3**. Nếu thay select bằng watch, số lần build sẽ là 4.

Bài 2. Viết lại giỏ hàng bằng ValueNotifier<List<CatalogItem>> với danh sách **bất biến**.

▼ Lời giải

```
class CartNotifier extends ValueNotifier<List<CatalogItem>> {
  CartNotifier() : super(const []);

  // List.unmodifiable: không ai sửa được list cũ → buộc phải tạo list mới (bất biến thật sự).
  void add(CatalogItem item) => value = List.unmodifiable([...value, item]);
  void removeAll() => value = const [];
  int get totalPrice => value.fold(0, (sum, i) => sum + i.price);
}
```

ValueNotifier so sánh value mới với cũ bằng ==; list mới → khác → báo listener. Test kiểm tra list cũ và mới **không** identical, và cart.value.add(...) ném `UnsupportedError`. Cách này giống cập nhật signal bằng `items.update(list => [...list, item])` trong Angular.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-30 (đọc file nguồn docs ở commit `ab59c61`):

- State management — intro — <https://docs.flutter.dev/data-and-backend/state-mgmt/intro> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/data-and-backend/state-mgmt/intro.md>
- Ephemeral vs app state — <https://docs.flutter.dev/data-and-backend/state-mgmt/ephemeral-vs-app> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/data-and-backend/state-mgmt/ephemeral-vs-app.md>
- Simple app state management (provider) — <https://docs.flutter.dev/data-and-backend/state-mgmt/simple> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/data-and-backend/state-mgmt/simple.md>
- State management options — <https://docs.flutter.dev/data-and-backend/state-mgmt/options> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/data-and-backend/state-mgmt/options.md>
- Tutorial: Use ChangeNotifier — <https://docs.flutter.dev/learn/pathway/tutorial/change-notifier> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/learn/pathway/tutorial/change-notifier.md>
- Tutorial: Use ListenableBuilder — <https://docs.flutter.dev/learn/pathway/tutorial/listenable-builder> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/learn/pathway/tutorial/listenable-builder.md>
- Architecture recommendations — <https://docs.flutter.dev/app-architecture/recommendations> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/app-architecture/recommendations.md>
- Case study — UI layer — <https://docs.flutter.dev/app-architecture/case-study/ui-layer> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/app-architecture/case-study/ui-layer.md>
- pub.dev: <https://pub.dev/packages/provider> · https://pub.dev/packages/flutter_riverpod · https://pub.dev/packages/flutter_bloc · <https://pub.dev/packages/signals>

Chương 2 — Bất đồng bộ: Future, async/await, Stream, isolate

Mục tiêu

- Dùng `Future` + `async/await`, chạy song song bằng `Future.wait`, xử lý lỗi và timeout.
- Tạo và dùng `Stream` (`async*/yield`, `StreamController`), hiển thị bằng `StreamBuilder`/`FutureBuilder`.
- Biết **event loop** của Dart và khi nào cần **isolate** (`compute`, `Isolate.run`) để UI không bị đứng.
- Tránh lỗi kinh điển: kết quả cũ về sau (race condition), `setState` sau `dispose`.

Giải thích đơn giản

RxJS / TypeScript	Dart
<code>Promise<T></code>	<code>Future<T></code>
<code>async/await</code>	<code>async/await</code> (giống hệt)
<code>Promise.all([...]) / forkJoin</code>	<code>Future.wait([...])</code>
<code>Observable<T></code>	<code>Stream<T></code>
<code>subscribe(next, error, complete)</code>	<code>listen(onData, onError:, onDone:) → trả StreamSubscription</code>
<code>unsubscribe()</code>	<code>subscription.cancel()</code>
<code>Subject / BehaviorSubject</code>	<code>StreamController / StreamController.broadcast()</code>
<code>map, filter, distinctUntilChanged, take</code>	<code>map, where, distinct, take</code>
<code>async pipe</code>	<code>FutureBuilder / StreamBuilder</code>
Web Worker	Isolate

Dart chạy code của bạn trên **một luồng** với **event loop** (vòng lặp sự kiện), giống JavaScript. `await` không chặn luồng: nó "tạm dừng" hàm và cho event loop làm việc khác (vẽ frame, xử lý chạm). Nhưng một vòng lặp tính toán nặng **không có await** sẽ chiếm luồng → UI đứng (jank). Khi đó dùng **isolate** — một "luồng" có bộ nhớ riêng.

Ví dụ

`examples/lib/chapters/ch02/async_demo.dart`:

Future

```
Future<String> fetchGreeting(String name, {Duration delay = const
Duration(milliseconds: 50)}) async {
  await Future<void>.delayed(delay);
  if (name.isEmpty) throw ArgumentError('Thiếu tên');
  return 'Xin chào $name';
}

/// Chạy song song và chờ tất cả (≈ Promise.all / forkJoin).
Future<List<String>> greetAll(List<String> names) => Future.wait([for (final n in
names) fetchGreeting(n)]);
```

Timeout: `await client.get(uri).timeout(const Duration(seconds: 8))` ném `TimeoutException` nếu quá hạn (dùng trong `DictionaryClient`, Chương 3).

Stream bằng `async*`

```
Stream<int> countdown(int from, {Duration interval = const Duration(seconds: 1)})
async* {
  for (var i = from; i >= 0; i--) {
    yield i;
    if (i > 0) await Future<void>.delayed(interval);
  }
}
```

`async*` + `yield` = hàm sinh (generator) trả về Stream — tiện hơn tự quản `StreamController`.

StreamBuilder

```
class _CountdownViewState extends State<CountdownView> {
  // Tạo stream MỘT lần trong State – nếu tạo trong build, mỗi lần build sẽ đếm lại từ
  đầu.
  late final Stream<int> _stream = countdown(widget.from);

  @override
  Widget build(BuildContext context) {
    return StreamBuilder<int>({
      stream: _stream,
      builder: (context, snapshot) => switch (snapshot) {
        AsyncSnapshot(connectionState: ConnectionState.done) => const Text('Hết
giờ!'),
        AsyncSnapshot(:final data?) => Text('Còn $data giây'),
        _ => const Text('Chuẩn bị...'),
      },
    );
  }
}
```

`AsyncSnapshot` có `connectionState` (`none/waiting/active/done`), `data`, `error`. `Pattern` `AsyncSnapshot(:final data?)` nghĩa là "có data khác null". `StreamBuilder` tự hủy đăng ký khi widget bị gỡ.

Isolate: compute

```
/// Hàm tốn CPU: đếm từ dài trong văn bản lớn.
int countLongWords(String text) => text.split(RegExp(r'\s+')).where((w) => w.length >= 8).length;

/// Chạy trong isolate khác để không làm đứng UI (trên web: chạy cùng luồng).
Future<int> countLongWordsInBackground(String text) => compute(countLongWords, text);
```

Docs `perf/isolates` gợi ý dùng `isolate` cho việc nặng như parse JSON lớn, xử lý ảnh, lọc danh sách rất dài. App TOEIC có thể cần khi nạp một file dữ liệu lớn lần đầu. Hàm truyền cho `compute/Isolate.run` phải là hàm top-level hoặc static (hoặc closure không bắt biến không gửi được), và dữ liệu được **sao chép** sang `isolate` kia.

Race condition trong ViewModel — "switchMap" bằng tay

Người dùng gõ "b" rồi "inv": nếu kết quả tìm "b" về **sau** "inv", danh sách hiển thị sai.

`WordListViewModel` giữ một số thứ tự yêu cầu và bỏ kết quả cũ:

```
Future<void> refresh() async {
  final id = ++_requestId;
  _loading = true;
  notifyListeners();
  try {
    final result = await _repository.search(query: _query, favoritesOnly:
    _favoritesOnly);
    if (id != _requestId) return; // đã có yêu cầu mới hơn → bỏ kết quả này
    _words = result;
    _error = null;
  } on Exception catch (e) {
    if (id != _requestId) return;
    _error = e;
  } finally {
    if (id == _requestId) {
      _loading = false;
      notifyListeners();
    }
  }
}
```

Test "bỏ qua kết quả Cũ về muộn (giống `switchMap`)" dùng một `Completer` để giữ yêu cầu đầu chậm lại (Chương 7).

Kết quả test thật

```
flutter test --reporter expanded test/chapters/ch02_test.dart (2026-09-30):
```

```
00:00 +0: Chương 2 – Future, Stream, isolate Future: await và Future.wait
00:00 +1: Chương 2 – Future, Stream, isolate Stream async*: countdown phát 3, 2, 1, 0 rồi đóng
00:00 +2: Chương 2 – Future, Stream, isolate StreamBuilder hiển thị từng giá trị
00:00 +3: Chương 2 – Future, Stream, isolate compute: chạy hàm nặng ở isolate khác
00:00 +4: Chương 2 – Future, Stream, isolate Bài 1: retry thử lại rồi thành công / hết lượt
thì ném lỗi
00:00 +5: Chương 2 – Future, Stream, isolate Bài 2: Completer biến callback thành Future
00:01 +6: All tests passed!
```

Đi sâu

Event loop: microtask và event

Dart có 2 hàng đợi: **microtask** (ưu tiên, ví dụ phần sau `await` của Future đã xong) và **event** (I/O, timer, chạm, vẽ frame). Học chi tiết không bắt buộc; chỉ nhớ: đừng tạo vòng lặp dài không `await` trên luồng UI.

Single-subscription vs broadcast

- Stream mặc định **single-subscription**: chỉ một người `listen` (giống Observable "lạnh" chỉ cho một subscriber).
- `StreamController.broadcast()`: nhiều người nghe cùng lúc (giống `Subject`). Người đến sau **không** nhận giá trị cũ (khác `BehaviorSubject`).

Lỗi trong Future/Stream

- `try { await f(); } on SocketException catch (e) {...}` — bắt theo kiểu.
- Future lỗi mà không `await/catchError` → lỗi "unhandled", đi tới `PlatformDispatcher.onError` (Tập 3, Monitoring).
- Lint `unawaited_futures` (dự án của sách bật) nhắc khi quên `await`; nếu cố ý không chờ, bọc `unawaited(f())`.

FutureBuilder và bẫy "tạo Future trong build"

`FutureBuilder(future: repo.load(), ...)` trong `build` → mỗi lần `build` **gọi lại** `load()`. Tạo Future một lần (trong `initState`, hoặc tốt hơn: trong `ViewModel`) rồi truyền vào. App mẫu dùng `ViewModel` + `Command` nên không cần `FutureBuilder`.

Lỗi và bẫy thường gặp

- Quên `await`** → code chạy tiếp trước khi xong; lỗi không bị bắt.

- `setState/notifyListeners` **SAU** `dispose` vì Future về muộn → kiểm tra `mounted`, hoặc bỏ qua theo `requestId`.
- **Tạo Stream/Future trong `build`**.
- **Không `cancel()` subscription** tự tạo → rõ ràng; `StreamBuilder` tự lo, còn `listen` thủ công thì phải hủy trong `dispose`.
- **Việc nặng trên luồng UI** → jank; dùng `compute`.
- **Kết quả cũ ghi đè kết quả mới (race)** → `requestId` / hủy yêu cầu cũ.

Tóm tắt

- `Future` $\approx$ `Promise`, `Stream` $\approx$ `Observable`, `async*` tạo stream, `StreamBuilder` $\approx$ `async pipe`.
- Một luồng + event loop; việc nặng → `isolate` (`compute`).
- Luôn nghĩ tới: hủy đăng ký, `mounted`, và thứ tự kết quả.

Bài tập (có lời giải)

Bài 1. Viết `retry(action, attempts, initialDelay)`: thử lại khi có `Exception`, thời gian chờ tăng gấp đôi mỗi lần (100 ms, 200 ms, 400 ms...), hết lượt thì ném lỗi cuối.

▼ Lời giải

`examples/lib/chapters/ch02/exercise_solution.dart`:

```
Future<T> retry<T>(
  Future<T> Function() action, {
  int attempts = 3,
  Duration initialDelay = const Duration(milliseconds: 100),
}) async {
  var delay = initialDelay;
  for (var i = 1; ; i++) {
    try {
      return await action();
    } on Exception {
      if (i >= attempts) rethrow;
      await Future<void>.delayed(delay);
      delay *= 2;
    }
  }
}
```

`rethrow` ném lại lỗi gốc (giữ stack trace). Test: hàm lỗi 2 lần rồi trả "ok" → nhận "ok" sau 3 lần gọi; hàm luôn lỗi với `attempts: 2` → `throwsException`. Tương đương `retry({count, delay})` của RxJS.

Bài 2. Một thư viện cũ chỉ có API callback `legacyLoad(key, (result, error) {...})`. Viết `loadAsFuture(key)` trả về `Future<String>`.

▼ Lời giải

```
Future<String> loadAsFuture(String key) {  
    final completer = Completer<String>();  
    legacyLoad(key, (result, error) {  
        if (error != null) {  
            completer.completeError(StateError('$error'));  
        } else {  
            completer.complete(result);  
        }  
    });  
    return completer.future;  
}
```

`Completer` giống `new Promise((resolve, reject) => ...)`. Test: `loadAsFuture('k')` → "giá trị của k"; `loadAsFuture('')` → `throwsStateError`. Plugin native (Tập 3) cũng hay dùng mẫu này.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-30:

- Dart — Asynchronous programming — <https://dart.dev/language/async> — <https://github.com/dart-lang/site-www/blob/001b59a9ba35a4f578c1dc6f922730c2cd413988/src/content/language/async.md>
- Dart — Async/await tutorial — <https://dart.dev/libraries/async/async-await> — <https://github.com/dart-lang/site-www/blob/001b59a9ba35a4f578c1dc6f922730c2cd413988/src/content/libraries/async/async-await.md>
- Dart — Using streams — <https://dart.dev/libraries/async/using-streams> — <https://github.com/dart-lang/site-www/blob/001b59a9ba35a4f578c1dc6f922730c2cd413988/src/content/libraries/async/using-streams.md>
- Dart — Creating streams — <https://dart.dev/libraries/async/creating-streams> — <https://github.com/dart-lang/site-www/blob/001b59a9ba35a4f578c1dc6f922730c2cd413988/src/content/libraries/async/creating-streams.md>
- Dart — Concurrency / isolates — <https://dart.dev/language/concurrency> — <https://github.com/dart-lang/site-www/blob/001b59a9ba35a4f578c1dc6f922730c2cd413988/src/content/language/concurrency.md>
- Flutter — Concurrency and isolates — <https://docs.flutter.dev/perf/isolates> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/perf/isolates.md>

- Flutter for React Native developers (Futures, async/await) — <https://docs.flutter.dev/flutter-for/react-native-devs> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/flutter-for/react-native-devs.md>

Chương 3 — HTTP và JSON

Mục tiêu

- Gọi API REST bằng package `http` (docs chính thức dùng package này trong Learning Pathway và cookbook).
- Chuyển JSON ↔ model Dart **bằng tay** (`jsonDecode`, `fromJson`), ép kiểu an toàn.
- Xử lý lỗi mạng: mã trạng thái, timeout, JSON hỏng, mất kết nối — trả về `Result` thay vì ném lỗi lung tung.
- Test HTTP **không cần mạng** bằng `MockClient`.
- Biết khi nào nên dùng code generation (`json_serializable`).

Giải thích đơn giản

Angular	Flutter
<code>HttpClient.get<T>(url)</code> trả <code>Observable<T></code>	<code>http.Client().get(uri)</code> trả <code>Future<Response></code>
Interceptor	Bọc <code>http.Client</code> (class con của <code>BaseClient</code>)
Kiểu <code>T</code> chỉ là "lời hứa" của TypeScript	Dart kiểm tra kiểu lúc chạy : phải ép <code>as String</code> đúng, sai là lỗi
<code>HttpTestingController</code>	<code>MockClient((request) async => Response(...))</code>

App "Sổ Từ Vựng" có nút **"Tra từ điển online"** gọi Free Dictionary API (`https://api.dictionaryapi.dev/api/v2/entries/en/<từ>`, định dạng theo README của dự án `meetDeveloper/freeDictionaryAPI`). App vẫn **offline-first**: dữ liệu chính nằm trong SQLite (Chương 4); mạng chỉ là phần thêm.

Ví dụ

Model từ JSON

```
examples/lib/data/services/dictionary_client.dart:
```

```

class Definition {
    const Definition({required this.partOfSpeech, required this.definition,
this.example});

    final String partOfSpeech;
    final String definition;
    final String? example;

    /// Ép kiểu cẩn thận: JSON từ mạng có thể thiếu trường.
    factory Definition.fromJson(String partOfSpeech, Map<String, Object?> json) =>
Definition(
    partOfSpeech: partOfSpeech,
    definition: json['definition'] as String? ?? '',
    example: json['example'] as String?,
    );
}

```

JSON của API là mảng lồng nhau: `entry` → `meanings[]` → `definitions[]`. Dùng collection `for` lồng nhau để "làm phẳng":

```

List<Definition> parseDefinitions(String body) {
    final decoded = jsonDecode(body);
    if (decoded is! List) throw const FormatException('Cần một mảng JSON');
    return [
        for (final entry in decoded.whereType<Map<String, Object?>>())
            for (final meaning in (entry['meanings'] as List? ?? const
[]).whereType<Map<String, Object?>>())
                for (final d in (meaning['definitions'] as List? ?? const
[]).whereType<Map<String, Object?>>())
                    Definition.fromJson(meaning['partOfSpeech'] as String? ?? '?', d),
    ];
}

```

`whereType<Map<String, Object?>>()` bỏ qua phần tử sai kiểu thay vì crash.

Client trả về Result

```
class DictionaryClient {
  DictionaryClient({http.Client? client, this.timeout = const Duration(seconds: 8)})
    : _client = client ?? http.Client();

  final http.Client _client;
  final Duration timeout;

  static Uri uriFor(String word) =>
    Uri.https('api.dictionaryapi.dev',
      '/api/v2/entries/en/${Uri.encodeComponent(word.trim())}');

  Future<Result<List<Definition>>> lookup(String word) async {
    try {
      final response = await _client.get(uriFor(word)).timeout(timeout);
      if (response.statusCode == 404) return const
Result.error(DictionaryException('Không tìm thấy từ này'));
      if (response.statusCode != 200) {
        return Result.error(DictionaryException('Lỗi máy chủ
(${response.statusCode}'));
      }
      return Result.ok(parseDefinitions(utf8.decode(response.bodyBytes)));
    } on TimeoutException {
      return const Result.error(DictionaryException('Hết thời gian chờ'));
    } on FormatException {
      return const Result.error(DictionaryException('Dữ liệu trả về không đúng định
dạng'));
    } on http.ClientException catch (e) {
      return Result.error(DictionaryException('Không kết nối được: ${e.message}'));
    }
  }
}
```

- `Result` (sealed class `Ok/Error`) là mẫu trong docs chính thức (`app-architecture/design-patterns/result`); file `lib/utils/result.dart` lấy nguyên mẫu đó (giấy phép BSD, có ghi công).
- `utf8.decode(response.bodyBytes)` thay vì `response.body`: an toàn khi server không gửi charset (chữ có dấu).
- `http.Client` được **tiêm qua constructor** → test thay bằng `MockClient` (docs cookbook "Mock dependencies" làm tương tự, nhưng dùng mockito + code generation; sách dùng `MockClient` có sẵn trong package `http`).

ViewModel dùng switch trên Result

Trong `WordDetailScreen`, kết quả của `vm.lookup` (một `Command0<List<Definition>>`) được hiển thị bằng `switch`:

```

switch (vm.lookup.result) {
  null => const SizedBox.shrink(),
  Ok(:final value) when value.isEmpty => const Text('Từ điển không có định nghĩa.'),
  Ok(:final value) => Column(/* 3 định nghĩa đầu */),
  Error(:final error) => Text('Không tra được: $error', style: TextStyle(color:
Theme.of(context).colorScheme.error)),
},

```

Test với MockClient — không cần mạng

```

test('200: tách định nghĩa, gọi đúng URL', () async {
  Uri? called;
  final client = DictionaryClient(
    client: MockClient((req) async {
      called = req.url;
      return http.Response(sampleJson, 200);
    }),
  );
  final result = await client.lookup('negotiate');
  expect(called.toString(),
'https://api.dictionaryapi.dev/api/v2/entries/en/negotiate');
  switch (result) {
    case Ok(:final value):
      expect(value, hasLength(2));
      expect(value.first.partOfSpeech, 'verb');
    case Error():
      fail('không mong đợi lỗi: $result');
  }
});

```

Kết quả thật (2026-09-30, `flutter test --reporter expanded test/data/services_test.dart`, phần HTTP):

```

00:00 +3: DictionaryClient (HTTP + JSON, MockClient) 200: tách định nghĩa, gọi đúng URL
00:00 +4: DictionaryClient (HTTP + JSON, MockClient) 404, 500, JSON hỏng, mất mạng →
Result.error với thông báo tiếng Việt
00:00 +5: DictionaryClient (HTTP + JSON, MockClient) hết thời gian chờ
00:00 +6: All tests passed!

```

Gọi API thật: NOT RUN — sandbox chặn `api.dictionaryapi.dev` (proxy từ chối kết nối). Định dạng JSON lấy từ README chính thức của dự án (đã mở); hành vi server thật (ví dụ mã 404 cho từ không có) là **UNVERIFIED**.

Đi sâu

Quyền mạng trên từng nền tảng

DOCS `data-and-backend/networking`: Android phải khai báo `<uses-permission android:name="android.permission.INTERNET" />` trong `AndroidManifest.xml` (bản debug tự có; bản release cần khai báo — dự án Tập 2 đã thêm). macOS cần entitlement mạng. iOS không cần khai báo cho HTTPS.

Code generation: khi nào?

DOCS `serialization/json`: **viết tay** cho dự án nhỏ; **code generation** (`json_serializable`, `built_value`) cho dự án vừa và lớn — đổi lại phải chạy `build_runner`. Kiến trúc chính thức còn gợi ý `freezed` cho model bất biến. Sách viết tay để thấy rõ cơ chế và để build không cần bước sinh code. App TOEIC có ít model (từ, cụm từ, bài đọc) → viết tay là đủ.

dio hay http?

`dio` (5.11.1, 2026-09-04) có interceptor, hủy request, upload tiến độ. `http` đơn giản và là lựa chọn của docs. Sách dùng `http`.

Hủy request khi rời màn hình

`http` không hủy request đang chạy được, nhưng ta có thể **bỏ qua kết quả** (như `requestId` ở Chương 2) hoặc dùng `Command` (chặn gọi trùng). Khi `ViewModel` bị dispose, không `notifyListeners()` nữa.

Lỗi và bẫy thường gặp

- `json['x'] as String` khi server trả `null` → `TypeError`. Dùng `as String?` + giá trị mặc định.
- Dùng `response.body` với tiếng Việt mà server không gửi charset → lỗi font chữ (mojibake). Dùng `utf8.decode(bodyBytes)`.
- Nối chuỗi URL bằng tay → quên encode ký tự đặc biệt. Dùng `Uri.https(host, path, query)`.
- Tạo `http.Client()` mới cho mỗi request mà không `close()` → tốn kết nối. Tạo một lần, thêm vào.
- Ném exception từ repository lên thẳng UI → màn hình đỏ. Bắt ở data layer, trả `Result`.
- Quên timeout → spinner quay mãi khi mạng chập chờn.

Tóm tắt

- `http` + `jsonDecode` + `fromJson` viết tay; ép kiểu an toàn (`as T?`, `whereType`).
- Bắt lỗi ở data layer, trả `Result<T>`; View dùng `switch` để hiển thị.
- Thêm `http.Client` → test bằng `MockClient`, không cần mạng.

Bài tập (có lời giải)

Bài 1. Từ JSON của Free Dictionary API, viết `firstAudioUrl(body)` trả về link mp3 phát âm đầu tiên **không rỗng**. API trả `"audio": "///ssl.gstatic.com/..."` (thiếu `https:`) — hãy thêm vào. Không có thì trả `null`.

▼ Lời giải

`examples/lib/chapters/ch03/exercise_solution.dart`:

```
String? firstAudioUrl(String body) {
  final decoded = jsonDecode(body);
  if (decoded is! List) return null;
  for (final entry in decoded.whereType<Map<String, Object?>>()) {
    for (final p in (entry['phonetics'] as List? ?? const
[]).whereType<Map<String, Object?>>()) {
      final audio = p['audio'] as String? ?? '';
      if (audio.isEmpty) continue;
      return audio.startsWith('///') ? 'https:$audio' : audio;
    }
  }
  return null;
}
```

Test (`ch03_ch04_test.dart`): JSON mẫu có một audio rỗng và một

`///ssl.example/negotiate.mp3` → nhận `https://ssl.example/negotiate.mp3`; {} hoặc `phonetics` rỗng → `null`.

Bài 2. Vì sao `DictionaryClient` nhận `http.Client?` trong constructor thay vì tự tạo `http.Client()` bên trong hàm `lookup`?

▼ Lời giải

(1) **Test**: truyền `MockClient` để kiểm tra URL, mã 404/500, JSON hỏng, mất mạng, timeout — không cần mạng thật (4 test trong `services_test.dart` làm đúng vậy). (2) **Tái sử dụng kết nối**: một client dùng chung giữ kết nối HTTP keep-alive. (3) **Mở rộng**: có thể truyền một client bọc thêm header/log (giống interceptor). Đây là DI bằng constructor — cùng tinh thần "Use dependency injection" của docs kiến trúc.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-30:

- Tutorial: Make HTTP requests — <https://docs.flutter.dev/learn/pathway/tutorial/http-requests> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/learn/pathway/tutorial/http-requests.md>
- Cookbook: Fetch data from the internet — <https://docs.flutter.dev/cookbook/networking/fetch-data> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/cookbook/networking/fetch-data.md>
- Networking — <https://docs.flutter.dev/data-and-backend/networking> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/data-and-backend/networking.md>
- JSON and serialization — <https://docs.flutter.dev/data-and-backend/serialization/json> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/data-and-backend/serialization/json.md>
- Cookbook: Mock dependencies using Mockito — <https://docs.flutter.dev/cookbook/testing/unit/mocking> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/cookbook/testing/unit/mocking.md>
- Design pattern: Result — <https://docs.flutter.dev/app-architecture/design-patterns/result> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/app-architecture/design-patterns/result.md>
- Mã nguồn mẫu Result (BSD): <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/examples/app-architecture/result/lib/result.dart>
- Free Dictionary API README: <https://github.com/meetDeveloper/freeDictionaryAPI>
- pub.dev: <https://pub.dev/packages/http> · <https://pub.dev/packages/dio> · https://pub.dev/packages/json_serializable

Chương 4 — Lưu trữ offline: `shared_preferences` và `SQLite (sqlite)`

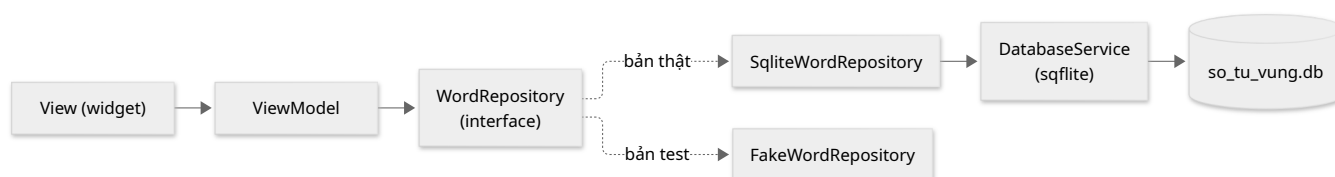
Mục tiêu

- Chọn đúng cách lưu: key-value (`shared_preferences`) cho cài đặt, **SQLite** (`sqlite`) cho dữ liệu có cấu trúc.
- Mở database, tạo bảng, **migration** (`onCreate` / `onUpgrade`), seed dữ liệu bằng `batch`, dùng `transaction`.
- Viết **repository** theo khuyến nghị kiến trúc: interface + bản SQLite + bản giả.
- Test với **SQLite thật** trên máy tính bằng `sqlite_common_ffi`; chạy SQLite trên **web** bằng `sqlite_common_ffi_web`.
- Tìm kiếm tiếng Việt "có dấu / không dấu" bằng cột `search_key`.

Giải thích đơn giản

Nhu cầu	Web / Angular	React Native (sách RN)	Flutter (sách dùng)
Cài đặt nhỏ (theme, giờ nhắc)	<code>localStorage</code>	<code>AsyncStorage</code>	<code>shared_preferences</code>
Dữ liệu có cấu trúc, tìm kiếm, thống kê	<code>IndexedDB</code>	<code>expo-sqlite</code>	<code>sqlite</code> (SQLite)
Bí mật (token, PIN)	—	<code>expo-secure-store</code>	<code>flutter_secure_storage</code> (Tập 3)

Docs chính thức: cookbook "Persist data with SQLite" dùng `sqlite` + `path`; "Store key-value data on disk" dùng `shared_preferences`; mẫu kiến trúc "offline-first" nói **repository là nguồn sự thật duy nhất** (single source of truth), và là nơi duy nhất được sửa dữ liệu.



Ví dụ

Mở database + migration — examples/lib/data/services/database_service.dart

```
class DatabaseService {
  DatabaseService._(this.db);

  final Database db;

  static const int schemaVersion = 2;
  static const String fileName = 'so_tu_vung.db';

  static Future<DatabaseService> open({DatabaseFactory? factory, String? path}) async {
    final f = factory ?? databaseFactory;
    final dbPath = path ?? (kIsWeb ? fileName : p.join(await f.getDatabasesPath(),
fileName));
    final db = await f.openDatabase(
      dbPath,
      options: OpenDatabaseOptions(
        version: schemaVersion,
        onConfigure: (db) => db.execute('PRAGMA foreign_keys = ON'),
        onCreate: (db, version) async {
          await createSchemaV1(db);
          await migrateToV2(db);
          await seed(db);
        },
        onUpgrade: (db, oldVersion, newVersion) async {
          if (oldVersion < 2) await migrateToV2(db);
        },
      ),
    );
    return DatabaseService._(db);
  }
  // createSchemaV1: CREATE TABLE words (...)
  // migrateToV2: CREATE TABLE reviews (... REFERENCES words(id) ON DELETE CASCADE)
  + index
}
```

- version + onCreate + onUpgrade = **migration**: máy mới cài chạy onCreate (tạo đủ mọi phiên bản); máy đang ở v1 chạy onUpgrade(1 → 2). Không bao giờ sửa code của phiên bản cũ đã phát hành — chỉ thêm bước mới.
- PRAGMA foreign_keys = ON: SQLite **mặc định tắt** khóa ngoại; phải bật mỗi lần mở.
- factory là tham số → test truyền databaseFactoryFfi, web dùng databaseFactoryFfiWeb.

Bảng words có cột search_key = "từ + nghĩa", viết thường, **bỏ dấu** — nhờ vậy gõ "dam phan" vẫn tìm được "đàm phán":

```
static String searchKey(String text, String meaning) => '$text
$meaning'.withoutAccents;
```

Seed 24 từ mẫu trong **một batch** (nhanh hơn nhiều so với 24 lần `insert` riêng lẻ):

```
static Future<void> seed(DatabaseExecutor db) async {
  final batch = db.batch();
  for (final (text, pos, meaning, example) in seedWords) {
    batch.insert('words', {
      'text': text, 'pos': pos, 'meaning': meaning, 'example': example,
      'search_key': searchKey(text, meaning),
    });
  }
  await batch.commit(noResult: true);
}
```

Repository — `sqlite_word_repository.dart`

```
@override
Future<List<Word>> search({String query = '', bool favoritesOnly = false}) async {
  final where = <String>[];
  final args = <Object?>[];
  final q = query.trim().withoutAccents;
  if (q.isNotEmpty) {
    where.add('w.search_key LIKE ?');
    args.add('"%q%"');
  }
  if (favoritesOnly) where.add('w.favorite = 1');
  final sql = StringBuffer(_select);
  if (where.isNotEmpty) sql.write(' WHERE ${where.join(' AND ')}');
  sql.write(' GROUP BY w.id ORDER BY w.text COLLATE NOCASE');
  final rows = await _db.rawQuery(sql.toString(), args);
  return rows.map(Word.fromRow).toList();
}
```

`_select` nối `words` với `reviews` (LEFT JOIN + COUNT/SUM) để mỗi `Word` có sẵn số lần ôn và số lần đúng. Giá trị người dùng nhập luôn đi qua **tham số ?** — không bao giờ nối vào chuỗi SQL (test "search an toàn với ký tự đặc biệt" thử `' OR 1=1 --`). Thứ tự "nên ôn trước" dùng điểm $(\text{đúng} + 1) / (\text{số lần ôn} + 2)$: từ hay sai < từ mới < từ đã thuộc.

Model đọc một dòng SQLite — SQLite không có kiểu bool nên lưu `0/1`:

```
factory Word.fromRow(Map<String, Object?> row) => Word(
  id: row['id']! as int,
  text: row['text']! as String,
  partOfSpeech: row['pos']! as String,
  meaning: row['meaning']! as String,
  example: (row['example'] as String?) ?? '',
  favorite: (row['favorite'] as int? ?? 0) == 1,
  reviewCount: row['review_count'] as int? ?? 0,
  correctCount: row['correct_count'] as int? ?? 0,
);
```

Cài đặt bằng `shared_preferences` — `key_value_store.dart` + `settings_repository.dart`

```
abstract interface class KeyValueStore {
  Future<String?> getString(String key);
  Future<void> setString(String key, String value);
  Future<void> remove(String key);
}

class SharedPreferencesStore implements KeyValueStore {
  SharedPreferencesStore([SharedPreferencesAsync? prefs]) : _prefs = prefs ??
  SharedPreferencesAsync();
  final SharedPreferencesAsync _prefs;
  // getString / setString / remove gọi thẳng _prefs
}
```

`SharedPreferencesAsync` là API mới của `shared_preferences` (mọi lời gọi async, không cache). `SettingsRepository` lưu theme (`ThemeMode.name`), giờ nhắc ("20:00") và tốc độ đọc; test dùng `MemoryStore` trong bộ nhớ.

Test với SQLite thật (FFI)

```
/// SQLite THẬT chạy trên máy tính (qua FFI), trong bộ nhớ – mỗi lần gọi là một DB mới.
Future<DatabaseService> openTestDatabase() {
  sqfliteFfiInit();
  return DatabaseService.open(factory: databaseFactoryFfi, path:
  inMemoryDatabasePath);
}
```

Test migration mở một **file** DB phiên bản 1 (chưa có bảng `reviews`), đóng lại, rồi mở bằng code mới:

```
final v1 = await databaseFactoryFfi.openDatabase(
  path,
  options: OpenDatabaseOptions(
    version: 1,
    onCreate: (db, _) async {
      await DatabaseService.createSchemaV1(db);
      await DatabaseService.seed(db);
    },
  ),
);
await v1.close();
final service = await DatabaseService.open(factory: databaseFactoryFfi, path: path);
// onUpgrade(1 → 2)
expect(await service.db.getVersion(), 2);
```

Kết quả thật (2026-09-30, `flutter test --reporter expanded test/data/database_test.dart`):

```
00:00 +0: DatabaseService (SQLite thật qua FFI) tạo mới: có bảng words + reviews và 24 từ mẫu
00:00 +1: DatabaseService (SQLite thật qua FFI) migration: file DB phiên bản 1 được nâng lên
2, dữ liệu cũ còn nguyên
00:00 +2: DatabaseService (SQLite thật qua FFI) khóa ngoại: xóa từ thì xóa luôn lịch sử ôn (ON
DELETE CASCADE)
00:00 +3: SqliteWordRepository search: theo từ, theo nghĩa có dấu và không dấu
00:00 +4: SqliteWordRepository search an toàn với ký tự đặc biệt (tham số ?, không nối chuỗi
SQL)
00:00 +5: SqliteWordRepository toggleFavorite + lọc yêu thích
00:00 +6: SqliteWordRepository recordReview + dueForReview ưu tiên từ hay sai / chưa ôn
00:00 +7: SqliteWordRepository stats: tổng, yêu thích, ôn hôm nay
00:00 +8: All tests passed!
```

SQLite trên web (Safari/Chrome)

`sqflite` chỉ có bản iOS/Android/macOS. Để bản web chạy được (con đường "iPhone không cần Mac" ở Tập 1, Chương 1), app dùng **import có điều kiện**:

```
// main.dart
import 'data/services/db_factory_stub.dart' if (dart.library.js_interop)
'data/services/db_factory_web.dart';
// ...
configureDatabaseFactory(); // web: databaseFactory = databaseFactoryFfiWeb;
iOS/Android: không làm gì
```

Web cần 2 file `web/sqlite3.wasm` và `web/sqflite_sw.js`, tạo bằng `dart run`

`sqflite_common_ffi_web:setup` (lệnh tải `sqlite3.wasm` bản `sqlite3-3.6.0` từ GitHub). Vì vậy `pubspec.yaml` ghim `sqlite3: 3.6.0` cho khớp. Docs kiến trúc gọi `sqflite_common_ffi_web` là plugin **experimental** (thử nghiệm).

Kiểm tra thật trong sandbox: bản build web mở trong Chromium headless hiện "24 từ" — tức SQLite (Wasm) đã tạo bảng và seed dữ liệu (`logs/check-all.txt`, "WEB SMOKE OK"); integration test trên Chrome cũng pass (Chương 7). Trên Safari iOS: **UNVERIFIED**.

Đi sâu

Đường dẫn file DB

`getDatabasesPath()` trả về thư mục riêng của app (iOS: thư mục Documents/Library của sandbox app; Android: thư mục databases). Gỡ app là mất DB. Muốn sao lưu, xuất file hoặc đồng bộ lên server.

Transaction

`db.transaction((txn) async {...})`: mọi lệnh bên trong thành công hết hoặc không lệnh nào có hiệu lực. **Bên trong** transaction phải dùng `txn`, không dùng `db` (dùng `db` sẽ bị treo — deadlock). Bài 2 minh họa.

Dữ liệu lớn có sẵn (app TOEIC)

App TOEIC có hàng nghìn từ vựng sẵn từ bộ dữ liệu (Task 5 dùng file JSON). Hai cách: (1) seed từ JSON asset lần đầu bằng `batch` trong `onCreate` (đơn giản; có thể chạy `jsonDecode` trong isolate); (2) đóng gói sẵn file `.db` trong `assets` và copy vào `getDatabasesPath()` lần đầu (nhanh nhất khi dữ liệu rất lớn). Cách (1) dễ migration hơn.

drift?

`drift` (2.35.0) cho truy vấn type-safe và stream tự cập nhật, nhưng cần code generation. Với app một người dùng, ít bảng, `sqflite` + `repository` là đủ và là lựa chọn của cookbook.

Lỗi và bẫy thường gặp

- Nối chuỗi SQL với dữ liệu người dùng → SQL injection, lỗi dấu nháy. Luôn dùng `?`.
- Quên bật `PRAGMA foreign_keys` → `ON DELETE CASCADE` không chạy.
- Sửa `onCreate` mà không tăng `version` → máy cũ không bao giờ nhận bảng mới.
- `ALTER TABLE ... ADD COLUMN ... NOT NULL` không có `DEFAULT` → lỗi với bảng đã có dữ liệu (Bài 1).
- Dùng db bên trong `transaction` → treo.
- Lưu dữ liệu lớn vào `shared_preferences` → chậm, không truy vấn được.
- Web thiếu `sqlite3.wasm/sqflite_sw.js` hoặc lệch phiên bản với gói `sqlite3` → lỗi khi mở DB trên web.
- Tìm kiếm không dấu bằng `LIKE` trên cột gốc → "dam phan" không khớp "đàm phán"; cần cột `search_key`.

Tóm tắt

- Cài đặt → `shared_preferences` (`SharedPreferencesAsync`); dữ liệu → `SQLite` (`sqflite`) qua `repository`.
- `version` + `onCreate` + `onUpgrade` cho migration; `batch` cho seed; `transaction` cho thao tác nhiều bước.
- Test bằng `SQLite` thật qua `sqflite_common_ffi`; web bằng `sqflite_common_ffi_web` + `import` có điều kiện.

Bài tập (có lời giải)

Bài 1. Viết migration phiên bản 3: thêm cột `level INTEGER (1..3)` cho bảng `words`. Các từ cũ phải có level 1.

▼ Lời giải

```
examples/lib/chapters/ch04/exercise_solution.dart:
```

```
Future<void> migrateToV3(Database db) async {
  await db.execute('ALTER TABLE words ADD COLUMN level INTEGER NOT NULL DEFAULT
1');
}
```

Trong `DatabaseService`: đổi `schemaVersion = 3`, thêm `await migrateToV3(db)`; vào cuối `onCreate`, và `if (oldVersion < 3) await migrateToV3(db)`; vào `onUpgrade`. Test mở DB v1 có dữ liệu, chạy `migrateToV3`, đọc lại "budget" → `level == 1`.

Bài 2. Viết `renameAll(db, {'cũ': 'mới', ...})` đổi tên nhiều từ; nếu một từ không tồn tại thì không đổi gì cả.

▼ Lời giải

```
Future<void> renameAll(Database db, Map<String, String> renames) {
  return db.transaction((txn) async {
    for (final e in renames.entries) {
      final n = await txn.rawUpdate('UPDATE words SET text = ? WHERE text = ?',
[e.value, e.key]);
      if (n == 0) throw StateError('Không có từ "${e.key}"');
    }
  });
}
```

Ném lỗi bên trong `transaction` → SQLite **rollback**. Test: `{'budget': 'Budget', 'khong-co': 'x'}` → ném `StateError` và "budget" vẫn còn nguyên; `{'budget': 'Budget'}` → đổi thành công.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-30 (đọc file nguồn docs ở commit `ab59c61`):

- Cookbook: Persist data with SQLite — <https://docs.flutter.dev/cookbook/persistence/sqlite> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/cookbook/persistence/sqlite.md>
- Cookbook: Store key-value data on disk — <https://docs.flutter.dev/cookbook/persistence/key-value> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/cookbook/persistence/key-value.md>
- Persistence — <https://docs.flutter.dev/data-and-backend/persistence> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/data-and-backend/persistence/index.md>
- Design pattern: Persistent storage architecture — SQL — <https://docs.flutter.dev/app-architecture/design-patterns/sql> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/app-architecture/design-patterns/sql.md>

780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/app-architecture/design-patterns/sql.md

- Design pattern: Key-value data — <https://docs.flutter.dev/app-architecture/design-patterns/key-value-data> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/app-architecture/design-patterns/key-value-data.md>
- Design pattern: Offline-first support — <https://docs.flutter.dev/app-architecture/design-patterns/offline-first> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/app-architecture/design-patterns/offline-first.md>
- pub.dev: <https://pub.dev/packages/sqflite> · https://pub.dev/packages/sqflite_common_ffi · https://pub.dev/packages/sqflite_common_ffi_web · https://pub.dev/packages/shared_preferences · <https://pub.dev/packages/drift>

Chương 5 — Animation: implicit, explicit, Hero

Mục tiêu

- Dùng **animation ngầm (implicit)**: `AnimatedContainer`, `TweenAnimationBuilder`, `AnimatedSwitcher` — chỉ đổi giá trị đích.
- Dùng **animation tường minh (explicit)**: `AnimationController` + `vsync` + `CurvedAnimation` + `AnimatedBuilder` — làm thẻ lật (flip card) cho màn hình ôn tập.
- Dùng **Hero** để chữ "bay" từ danh sách sang màn hình chi tiết.
- Test animation bằng `pump(duration)` và biết vì sao `pumpAndSettle` có thể treo.

Giải thích đơn giản

Docs ("Introduction to animations") chia hai loại:

Loại	Bạn làm gì	Widget	Giống Angular
Implicit (ngầm)	Đổi thuộc tính → widget tự chạy từ giá trị cũ tới mới	<code>AnimatedContainer</code> , <code>AnimatedOpacity</code> , <code>TweenAnimationBuilder</code> , <code>AnimatedSwitcher</code>	CSS <code>transition</code>
Explicit (tường minh)	Tự điều khiển <code>AnimationController</code> : <code>forward</code> , <code>reverse</code> , <code>repeat</code> , <code>stop</code>	<code>AnimationController</code> + <code>AnimatedBuilder</code> / <code>*Transition</code>	<code>@angular/animations</code> với <code>AnimationPlayer</code>

Quy tắc chọn: thử implicit trước; chỉ dùng explicit khi cần lập, điều khiển thủ công, hoặc phối hợp nhiều animation.

Ví dụ

Implicit: thanh tiến độ và hộp đối kích thước

```
examples/lib/chapters/ch05/animation_demo.dart:
```

```

class AnimatedProgress extends StatelessWidget {
  const AnimatedProgress({super.key, required this.value});

  final double value; // 0..1

  @override
  Widget build(BuildContext context) {
    final scheme = Theme.of(context).colorScheme;
    return TweenAnimationBuilder<double>(
      tween: Tween(end: value.clamp(0, 1)),
      duration: const Duration(milliseconds: 500),
      curve: Curves.easeOut,
      builder: (context, v, _) => Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          LinearProgressIndicator(value: v, minHeight: 10, color: scheme.primary),
          Text('${(v * 100).round()}%'),
        ],
      ),
    );
  }
}

```

`TweenAnimationBuilder` chỉ cần giá trị `end`; khi `value` đổi, nó chạy từ giá trị **hiện tại** tới giá trị mới.

```

AnimatedContainer(
  duration: const Duration(milliseconds: 300),
  curve: Curves.easeInOut,
  height: _big ? 140 : 70,
  decoration: BoxDecoration(
    color: _big ? scheme.tertiaryContainer : scheme.primaryContainer,
    borderRadius: BorderRadius.circular(_big ? 32 : 8),
  ),
  alignment: Alignment.center,
  child: const Text('Chạm để đổi'),
)

```

Explicit: thẻ lật — examples/lib/ui/core/flip_card.dart

```
class _FlipCardState extends State<FlipCard> with SingleTickerProviderStateMixin {
  // SingleTickerProviderStateMixin cung cấp "vsync": animation chỉ chạy khi màn hình
  đang vẽ.
  late final AnimationController _controller = AnimationController(
    vsync: this,
    duration: widget.duration,
    value: widget.flipped ? 1 : 0,
  );
  late final Animation<double> _angle = CurvedAnimation(parent: _controller, curve:
Curves.easeInOut);

  @override
  void didUpdateWidget(FlipCard oldWidget) {
    super.didUpdateWidget(oldWidget);
    if (oldWidget.flipped != widget.flipped) {
      widget.flipped ? _controller.forward() : _controller.reverse();
    }
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return AnimatedBuilder(
      animation: _angle,
      builder: (context, _) {
        final angle = _angle.value * math.pi;
        final showBack = angle > math.pi / 2;
        return Transform(
          alignment: Alignment.center,
          transform: Matrix4.identity()
            ..setEntry(3, 2, 0.001) // phối cảnh (perspective)
            ..rotateY(angle),
          child: showBack
            ? Transform(alignment: Alignment.center, transform:
Matrix4.rotationY(math.pi), child: widget.back)
            : widget.front,
        );
      },
    );
  }
}
```

- `AnimationController` chạy giá trị $0 \rightarrow 1$ trong `duration`. `CurvedAnimation` làm chuyển động mượt (ease-in-out).
- `AnimatedBuilder` build lại **chỉ phần bên trong** mỗi frame.

- Nửa đường (90°) thì đổi sang mặt sau; mặt sau được lật thêm 180° để chữ không bị ngược.
- Màn hình **Ôn tập** của app dùng `FlipCard(flipped: vm.revealed, ...): ViewModel` đổi `revealed`, widget tự lật.

Hero: chữ bay từ danh sách sang chi tiết

```
// Danh sách (word_list_screen.dart)
Hero(tag: 'word-${word.id}', child: Material(type: MaterialType.transparency, child:
Text(word.text)))

// Chi tiết (word_detail_screen.dart)
Hero(tag: 'word-${word.id}', child: Material(type: MaterialType.transparency, child:
Text(word.text, style: text.displaySmall)))
```

Hai `Hero` cùng `tag` ở hai route → Flutter tự vẽ chữ bay và phóng to khi chuyển trang. Bọc `Material` trong suốt để chữ không bị gạch chân vàng khi đang bay (vì lúc bay không có `Material` tổ tiên).

Test animation

```
testWidgets('AnimatedProgress chạy từ giá trị cũ tới mới trong 500ms', (tester) async
{
  await pumpApp(tester, const AnimatedProgress(value: 0.2));
  await tester.pumpAndSettle();
  expect(find.text('20%'), findsOneWidget);
  await pumpApp(tester, const AnimatedProgress(value: 1));
  await tester.pump(const Duration(milliseconds: 100));
  final mid = tester.widget<LinearProgressIndicator>
(find.byType(LinearProgressIndicator)).value!;
  expect(mid, inExclusiveRange(0.2, 1.0)); // đang ở giữa đường
  await tester.pumpAndSettle();
  expect(find.text('100%'), findsOneWidget);
});
```

Kết quả thật (2026-09-30, `flutter test --reporter expanded test/chapters/ch05_test.dart`):

```
00:00 +0: Chương 5 – animation AnimatedProgress chạy từ giá trị cũ tới mới trong 500ms
00:00 +1: Chương 5 – animation FlipCard: nửa đường thì đổi mặt
00:00 +2: Chương 5 – animation Bài 1: PulsingDot lặp khi active, dừng khi tắt
00:00 +3: Chương 5 – animation Bài 2: AnimatedScore có cả chữ cũ và mới trong lúc chuyển
00:00 +4: All tests passed!
```

Đi sâu

pump VS pumpAndSettle

- `pump(d)`: cho thời gian trôi `d` rồi vẽ một frame — dùng để kiểm tra trạng thái **giữa** animation.

- `pumpAndSettle()` : vẽ liên tục tới khi hết frame cần vẽ. Nếu có animation **lặp vô hạn** (`repeat()`), nó **không bao giờ xong** và báo lỗi "pumpAndSettle timed out". Sách gặp thật lỗi này khi test tab Lab (có `PulsingDot`) và sửa bằng `pump(...)` thay vì `pumpAndSettle()` (xem `test/app_test.dart`, tham số `settle`).

vsync và Ticker

`vsync: this` (từ `SingleTickerProviderStateMixin`) nối controller với **Ticker** — gọi callback mỗi frame, và tự dừng khi widget không hiển thị (ví dụ tab khác). Nhiều controller trong một State → `TickerProviderStateMixin`.

Hiệu năng

Animation chạy 60–120 frame/giây: phần build trong `AnimatedBuilder` phải nhẹ. Truyền phần không đổi vào tham số `child` của `AnimatedBuilder` để khỏi build lại. Dùng `*Transition` (`FadeTransition`, `ScaleTransition`) khi có thể — chúng chỉ đổi thuộc tính vẽ, không build lại cây.

Giảm chuyển động

Người dùng bật "Giảm chuyển động" (iOS: Reduce Motion) →

`MediaQuery.disableAnimationsOf(context)` trả `true`; nên rút ngắn hoặc tắt animation không cần thiết.

Lỗi và bẫy thường gặp

- **Quên `dispose()` `AnimationController`** → Ticker rò rỉ, test báo lỗi.
- **Tạo `AnimationController` trong `build`.**
- **`pumpAndSettle` với animation lặp** → timeout.
- **`AnimatedSwitcher` không chạy** vì con mới và cũ cùng loại, cùng key → thêm `key: ValueKey(giá trị)` (Bài 2).
- **Hero trùng tag** trên cùng một màn hình → lỗi "There are multiple heroes that share the same tag".
- **Build nặng trong `AnimatedBuilder`** → giật.

Tóm tắt

- Implicit trước (`AnimatedContainer`, `TweenAnimationBuilder`, `AnimatedSwitcher`); explicit khi cần điều khiển.
- Explicit = `AnimationController` + `vsync` + `CurvedAnimation` + `AnimatedBuilder`; luôn `dispose`.
- Hero cho chuyển trang đẹp. Test bằng `pump(d)`; cẩn thận `pumpAndSettle` với animation lặp.

Bài tập (có lời giải)

Bài 1. Làm `PulsingDot`: chấm đỏ nhấp nháy lặp lại khi `active = true`, dừng khi `false`.

▼ Lời giải

`examples/lib/chapters/ch05/exercise_solution.dart` (phần chính):

```
class _PulsingDotState extends State<PulsingDot> with
SingleTickerProviderStateMixin {
  late final AnimationController _controller = AnimationController(
    vsync: this,
    duration: const Duration(milliseconds: 600),
    lowerBound: 0.3,
  );

  @override
  void initState() {
    super.initState();
    if (widget.active) _controller.repeat(reverse: true);
  }

  @override
  void didUpdateWidget(PulsingDot oldWidget) {
    super.didUpdateWidget(oldWidget);
    if (widget.active != oldWidget.active) {
      widget.active ? _controller.repeat(reverse: true) : _controller.stop();
    }
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) => FadeTransition(
    opacity: _controller,
    child: Icon(Icons.circle, color: Theme.of(context).colorScheme.error, size:
16),
  );
}
```

Test: `tester.hasRunningAnimations` là `true` khi `active`, `false` sau khi đổi sang `active: false`, và khi đó `pumpAndSettle()` xong bình thường.

Bài 2. Điểm số đổi thì chữ cũ mờ đi, chữ mới hiện lên.

▼ Lời giải

```

AnimatedSwitcher(
  duration: const Duration(milliseconds: 250),
  // Key khác nhau → AnimatedSwitcher biết đây là widget MỚI và chạy animation
  chuyển.
  child: Text('$score điểm', key: ValueKey(score), style:
Theme.of(context).textTheme.headlineMedium),
)

```

Test: đổi 10 → 20, sau 100 ms cả "10 điểm" và "20 điểm" cùng có trong cây (đang chuyển); sau `pumpAndSettle` chỉ còn "20 điểm". Bỏ `key` đi thì test đầu sẽ thất bại vì không có animation.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-30 (đọc file nguồn docs ở commit `ab59c61`):

- Tutorial: Add implicit animations — <https://docs.flutter.dev/learn/pathway/tutorial/implicit-animations> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/learn/pathway/tutorial/implicit-animations.md>
- Introduction to animations — <https://docs.flutter.dev/ui/animations> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/ui/animations/index.md>
- Implicit animations — <https://docs.flutter.dev/ui/animations/implicit-animations> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/ui/animations/implicit-animations.md>
- Animations tutorial (explicit) — <https://docs.flutter.dev/ui/animations/tutorial> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/ui/animations/tutorial.md>
- Hero animations — <https://docs.flutter.dev/ui/animations/hero-animations> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/ui/animations/hero-animations.md>

Chương 6 — Device APIs: đọc to (text-to-speech) và thông báo cục bộ

Mục tiêu

- Thêm **plugin** (package có code native) và hiểu cấu hình theo nền tảng.
- Đọc to từ vừng bằng `flutter_tts`: ngôn ngữ, tốc độ, chế độ im lặng trên iPhone.
- Đặt **thông báo cục bộ hằng ngày** bằng `flutter_local_notifications` + `timezone` + `flutter_timezone`: khởi tạo, **xin quyền đúng lúc**, `zonedSchedule`, hủy.
- Bọc plugin sau **interface** (`TtsService`, `ReminderService`) để test không cần thiết bị.
- Biết giới hạn trên web.

Giải thích đơn giản

Plugin = package Dart + code native (Swift/Kotlin/JS). Bạn gọi hàm Dart; plugin chuyển lời gọi sang API của hệ điều hành qua **platform channel** (Tập 3, Chương 3). Docs "Using packages" dạy cách thêm: `flutter pub add <tên>`. Hai package của chương này **không** được docs nêu tên — đây là lựa chọn của sách (xem [STACK.md](#) mục 5):

Việc	Package (phiên bản ghim)	iOS dùng	Android dùng	Web
Đọc to	<code>flutter_tts</code> 4.2.5	<code>AVSpeechSynthesizer</code>	<code>TextToSpeech</code>	Web Speech API
Thông báo theo lịch	<code>flutter_local_notifications</code> 22.3.1	<code>UserNotifications</code>	<code>AlarmManager</code> + <code>NotificationManager</code>	Không có lịch
Múi giờ	<code>timezone</code> 0.11.1, <code>flutter_timezone</code> 5.1.0	—	—	—

So với React Native (sách RN dùng `expo-speech`, `expo-notifications`): ý tưởng giống; khác là Flutter không có "Expo Go", nên plugin nào cũng dùng được ngay khi build app (nhưng phải build lại app sau khi thêm plugin — hot reload không đủ).

Nguyên tắc kiến trúc: UI và ViewModel **không** gọi plugin trực tiếp. Chúng gọi interface; bản thật bọc plugin, bản giả dùng khi test.

Ví dụ

TTS — examples/lib/data/services/tts_service.dart

```
abstract interface class TtsService {
  Future<void> speak(String text);
  Future<void> stop();
  Future<void> setRate(double rate);
}

class FlutterTtsService implements TtsService {
  FlutterTtsService([FlutterTts? tts]) : _tts = tts ?? FlutterTts();

  final FlutterTts _tts;
  bool _ready = false;

  Future<void> _ensureReady() async {
    if (_ready) return;
    await _tts.setLanguage('en-US');
    await _tts.awaitSpeakCompletion(true);
    if (!kIsWeb && defaultTargetPlatform == TargetPlatform.iOS) {
      // Đọc được cả khi iPhone bật chế độ im lặng (README của flutter_tts:
      // setIosAudioCategory).
      await _tts.setIosAudioCategory(IosTextToSpeechAudioCategory.playback, [
        IosTextToSpeechAudioCategoryOptions.mixWithOthers,
      ]);
    }
    _ready = true;
  }

  @override
  Future<void> speak(String text) async {
    await _ensureReady();
    await _tts.stop();
    await _tts.speak(text);
  }
  // stop(), setRate() ...
}
```

- `awaitSpeakCompletion(true)`: `speak` chỉ hoàn thành khi đọc xong → nút có thể hiện "Đang đọc...".
- Tốc độ đọc (`setSpeechRate`) được lưu trong Cài đặt (0.2–1.0).
- Widget dùng qua provider: `context.read<TtsService>().speak(word.text)`.

Nút phát âm tái sử dụng — examples/lib/chapters/ch06/speak_button.dart:

```
Future<void> _speak() async {
  setState(() => _speaking = true);
  try {
    await context.read<TtsService>().speak(widget.text);
  } finally {
    if (mounted) setState(() => _speaking = false);
  }
}
```

Thông báo hằng ngày — examples/lib/data/services/reminder_service.dart

```
/// Lần tới của giờ [time] tính từ [now] (hôm nay nếu chưa qua, không thì ngày mai).
Hàm thuần – dễ test.
tz.TZDateTime nextInstanceOf(TimeOfDay time, tz.TZDateTime now) {
  var scheduled = tz.TZDateTime(now.location, now.year, now.month, now.day, time.hour,
time.minute);
  if (!scheduled.isAfter(now)) scheduled = scheduled.add(const Duration(days: 1));
  return scheduled;
}
```

Khởi tạo (một lần): nạp dữ liệu múi giờ, lấy múi giờ của máy, khởi tạo plugin **không xin quyền ngay**:

```
Future<void> _init() async {
  if (_initialized) return;
  tzdata.initializeTimeZones();
  final info = await FlutterTimezone.getLocalTimezone();
  tz.setLocalLocation(tz.getLocation(info.identifier));
  await _plugin.initialize(
    settings: const InitializationSettings(
      android: AndroidInitializationSettings('@mipmap/ic_launcher'),
      // Không xin quyền ngay khi mở app; xin khi người dùng bật nhắc
      (requestPermission).
      iOS: DarwinInitializationSettings(
        requestAlertPermission: false,
        requestBadgePermission: false,
        requestSoundPermission: false,
      ),
    ),
  );
  _initialized = true;
}
```

Xin quyền khi người dùng **bật** công tắc "Nhắc ôn hằng ngày", rồi đặt lịch lặp mỗi ngày:

```

@override
Future<void> scheduleDaily(TimeOfDay time) async {
  if (!isSupported) return;
  await _init();
  await _plugin.zonedSchedule(
    id: _id,
    title: 'Đến giờ ôn từ vựng',
    body: 'Dành 5 phút ôn lại các từ hôm nay nhé!',
    scheduledDate: nextInstanceOf(time, tz.TZDateTime.now(tz.local)),
    notificationDetails: const NotificationDetails(
      android: AndroidNotificationDetails('daily_review', 'Nhắc ôn hằng ngày'),
      iOS: DarwinNotificationDetails(),
    ),
    androidScheduleMode: AndroidScheduleMode.inexactAllowWhileIdle,
    matchDateTimeComponents: DateTimeComponents.time, // lặp lại mỗi ngày cùng giờ
  );
}

```

Ở phiên bản 22, các hàm của plugin dùng **tham số có tên** (`initialize(settings: ...)`, `cancel(id: ...)`, `zonedSchedule(id: ..., scheduledDate: ...)`) — nhiều bài viết cũ trên mạng dùng tham số vị trí sẽ không biên dịch.

`SettingsViewModel.setReminder` quyết định luồng: web → báo "Bản web không hỗ trợ nhắc theo lịch"; không có quyền → báo hướng dẫn bật trong Cài đặt; có quyền → đặt lịch + lưu giờ.

Cấu hình native đã thêm vào dự án (theo README của package)

- **iOS** — `ios/Runner/AppDelegate.swift`: `UNUserNotificationCenter.current().delegate = self as? UNUserNotificationCenterDelegate` để hiện thông báo cả khi app đang mở.
- **Android** — `android/app/build.gradle.kts`: `isCoreLibraryDesugaringEnabled = true + coreLibraryDesugaring("com.android.tools:desugar_jdk_libs:2.1.4")`; `AndroidManifest.xml`: quyền `RECEIVE_BOOT_COMPLETED` và hai `<receiver>` `ScheduledNotificationReceiver`, `ScheduledNotificationBootReceiver`.
- **Build iOS/Android với cấu hình này: NOT RUN** (không có Xcode/Android SDK trong sandbox).

Test không cần thiết bị

```

testWidgets('SpeakButton gọi TtsService lấy từ provider', (tester) async {
  final tts = FakeTts();
  await pumpApp(tester, Provider<TtsService>.value(value: tts, child: const
SpeakButton(text: 'deadline')));
  await tester.tap(find.text('Đọc "deadline"'));
  await tester.pump();
  expect(tts.spoken, ['deadline']);
});

```

Test lịch nhắc dùng múi giờ thật `Asia/Ho_Chi_Minh` (19:00 → nhắc 20:00 cùng ngày; nhắc 19:00 → ngày mai). Kết quả thật (2026-09-30):

```
00:00 +0: Chương 6 – device APIs SpeakButton gọi TtsService lấy từ provider
00:00 +1: Chương 6 – device APIs Bài 1: nextWeeklyInstance – thứ Hai tới lúc 8:00
00:00 +2: All tests passed!
```

và trong `test/data/services_test.dart`: "nextInstanceOf (lịch nhắc hằng ngày) hôm nay nếu chưa tới giờ, ngày mai nếu đã qua" — pass.

Đọc to và thông báo trên iPhone/Android thật: NOT RUN. Trên web (Chromium headless) TTS không được kiểm tra bằng tai; test chỉ chứng minh app gọi đúng service.

Đi sâu

Xin quyền: khi nào và thế nào?

- iOS: hỏi quyền **một lần**; người dùng từ chối thì chỉ bật lại được trong Settings. Vì vậy: chỉ hỏi khi người dùng chủ động bật nhắc, và giải thích trước lợi ích.
- Android 13+: cần quyền `POST_NOTIFICATIONS` (plugin khai báo sẵn), hỏi bằng `requestNotificationsPermission()`. Thông báo **đúng từng phút** (exact alarm) cần thêm quyền và chính sách của Play Store — nhắc ôn bài không cần chính xác tuyệt đối nên sách dùng `inexactAllowWhileIdle`.
- iOS chỉ giữ tối đa 64 thông báo đang chờ (README của package) — dùng lịch lặp (`matchDateTimeComponents`) thay vì đặt 365 thông báo.

Múi giờ

`zonedSchedule` cần `TZDateTime`. Nếu quên `tz.setLocalLocation(...)`, `tz.local` mặc định là UTC → nhắc lệch 7 tiếng ở Việt Nam.

TTS: giọng và ngôn ngữ

`getLanguages`, `getVoices`, `setVoice` cho phép chọn giọng (ví dụ en-US / en-GB). Máy có thể thiếu giọng; kiểm tra `isLanguageAvailable('en-US')`. Trên web, danh sách giọng phụ thuộc trình duyệt.

Web

`flutter_local_notifications` trên web không đặt lịch được (sách tắt tính năng qua `isSupported`). `flutter_tts` trên web dùng Web Speech API của trình duyệt — Safari iOS có hỗ trợ nhưng hành vi cụ thể **UNVERIFIED**.

Lỗi và bẫy thường gặp

- **Hot reload sau khi thêm plugin** → lỗi `MissingPluginException`. Dừng app và `flutter run` lại.
- **Xin quyền ngay khi mở app** → người dùng từ chối theo phản xạ.
- **Quên múi giờ** → giờ nhắc lệch.
- **Gọi plugin trong test** → `MissingPluginException`; dùng fake qua interface.
- **Không có desugaring (Android)** → build lỗi với `flutter_local_notifications 22` (theo README; NOT RUN trong sandbox).
- **iPhone ở chế độ im lặng** → TTS không kêu nếu không đặt audio category `playback`.

Tóm tắt

- Plugin = Dart + native. Bọc sau interface; test bằng fake.
- TTS: `flutter_tts` (`setLanguage`, `setSpeechRate`, `awaitSpeakCompletion`, iOS audio category).
- Nhắc hằng ngày: `timezone` + `flutter_timezone` + `zonedSchedule(... matchDateTimeComponents: time)`; xin quyền khi bật.
- Web: có TTS, không có lịch thông báo.

Bài tập (có lời giải)

Bài 1. Viết `nextWeeklyInstance(time, weekday, now)` cho nhắc "mỗi thứ Hai 8:00" (dùng với `DateTimeComponents.dayOfWeekAndTime`).

▼ Lời giải

`examples/lib/chapters/ch06/exercise_solution.dart`:

```
tz.TZDateTime nextWeeklyInstance(TimeOfDay time, int weekday, tz.TZDateTime now) {
  var d = tz.TZDateTime(now.location, now.year, now.month, now.day, time.hour,
    time.minute);
  while (d.weekday != weekday || !d.isAfter(now)) {
    d = tz.TZDateTime(d.location, d.year, d.month, d.day + 1, time.hour,
      time.minute);
  }
  return d;
}
```

Tạo lại `TZDateTime` với `day + 1` (thay vì `add(Duration(days: 1))`) để giữ đúng giờ khi có đổi giờ mùa hè ở múi giờ khác. Test: thứ Tư 30/9/2026 10:00 → thứ Hai 5/10/2026 8:00; thứ Tư 9:00 (đã qua) → thứ Tư tuần sau 7/10.

Bài 2. Vì sao `SettingsViewModel` nhận `ReminderService` (interface) thay vì `FlutterLocalNotificationsPlugin`? Viết một test kiểm tra "bật nhắc → xin quyền đúng 1 lần → đặt lịch lúc 20:00".

▼ Lời giải

Vì plugin cần thiết bị thật và hộp thoại quyền của hệ điều hành; trong test ta cần điều khiển kết quả (cho phép / từ chối / web). Test dùng **mocktail** để kiểm tra lời gọi (`test/ui/viewmodels_test.dart`):

```
final reminders = MockReminderService();
when(() => reminders.isSupported).thenReturn(true);
when(reminders.requestPermission).thenAnswer((_) async => true);
when(() => reminders.scheduleDaily(any())).thenAnswer((_) async {});
final vm = SettingsViewModel(settings: SettingsRepository(MemoryStore()),
reminders: reminders, tts: FakeTts());
await vm.setReminder(const TimeOfDay(hour: 20, minute: 0));
verify(reminders.requestPermission).called(1);
verify(() => reminders.scheduleDaily(const TimeOfDay(hour: 20, minute:
0))).called(1);
expect(vm.message, 'Sẽ nhắc lúc 20:00 mỗi ngày');
```

`registerFallbackValue(const TimeOfDay(hour: 0, minute: 0))` (trong `setUpAll`) cần cho `any()` với kiểu không phải cơ bản.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-30:

- Using packages — <https://docs.flutter.dev/packages-and-plugins/using-packages> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/packages-and-plugins/using-packages.md>
- Platform channels (plugin hoạt động thế nào) — <https://docs.flutter.dev/platform-integration/platform-channels> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/platform-integration/platform-channels.md>
- flutter_tts 4.2.5 (README đọc trong pub cache): https://pub.dev/packages/flutter_tts
- flutter_local_notifications 22.3.1 (README: iOS setup, Android Gradle/Manifest, giới hạn 64 thông báo iOS): https://pub.dev/packages/flutter_local_notifications
- timezone 0.11.1: <https://pub.dev/packages/timezone> · flutter_timezone 5.1.0: https://pub.dev/packages/flutter_timezone
- mocktail 1.0.5: <https://pub.dev/packages/mocktail>

Chương 7 — Testing: unit, widget, integration; fake và mock

Mục tiêu

- Hiểu 3 loại test của Flutter và khi nào dùng loại nào (bảng đánh đổi của docs).
- Viết **unit test** cho repository/ViewModel bằng **fake**; dùng **mocktail** khi cần `verify` lời gọi.
- Viết **widget test** cho màn hình và cho **toàn app** (router + provider thật, dữ liệu giả).
- Viết và chạy **integration test** (`integration_test`) — trong sandbox đã chạy thật trên Chrome headless.
- Tránh các bẫy: thời gian giả, `pumpAndSettle` treo, State bị giữ giữa các lần pump.

Giải thích đơn giản

Docs "Testing Flutter apps": app được test tốt có **nhiều** unit test và widget test, cộng **đủ** integration test cho các luồng quan trọng. Bảng đánh đổi (tóm tắt từ docs):

	Unit	Widget	Integration
Độ tin cậy	Thấp	Cao hơn	Cao nhất
Chi phí bảo trì	Thấp	Cao hơn	Cao nhất
Phụ thuộc	Ít	Nhiều hơn	Nhiều nhất
Tốc độ	Nhanh	Nhanh	Chậm

Angular	Flutter
Jasmine/Jest describe/it	group / test (package test, có trong flutter_test)
TestBed + component test	testWidgets + WidgetTester
jasmine.createSpyObj	fake tự viết, hoặc mocktail (Mock, when, verify)
fakeAsync / tick	tester.pump(Duration) (trong testWidgets thời gian là giả)
Cypress/Playwright E2E	integration_test (+ flutter drive trên web)

Kiến trúc chính thức: "Test architectural components separately, and together" và "Make fakes for testing". App "Số Từ Vựng" có: fake cho repository, TTS, nhắc nhở (`test/fakes/fakes.dart`); test data layer với **SQLite thật** qua FFI; test ViewModel không cần widget; test toàn app bằng widget test; một integration test chạy app thật.

Ví dụ

Fake — bản cài đặt đơn giản trong bộ nhớ

```
class FakeTts implements TtsService {  
  final List<String> spoken = [];  
  double rate = 0.5;  
  
  @override  
  Future<void> speak(String text) async => spoken.add(text);  
  
  @override  
  Future<void> stop() async {}  
  
  @override  
  Future<void> setRate(double rate) async => this.rate = rate;  
}
```

`FakeWordRepository` còn có "cổng" `searchGate` (một `Completer`) để giả lập kết quả về **chậm** — dùng cho test race condition.

Unit test ViewModel: bỏ qua kết quả cũ (giống `switchMap`)

```
test('bỏ qua kết quả cũ về muộn (giống switchMap)', () async {  
  final repo = FakeWordRepository();  
  final vm = WordListViewModel(repository: repo);  
  await pumpEventQueue();  
  final slow = Completer<void>();  
  repo.searchGate = slow;  
  final first = vm.search('b'); // chờ cổng  
  repo.searchGate = null;  
  await vm.search('inv'); // yêu cầu mới xong trước  
  expect(vm.words.single.text, 'invoice');  
  slow.complete();  
  await first; // kết quả cũ ('budget') về sau → phải bị bỏ  
  expect(vm.words.single.text, 'invoice');  
  expect(vm.loading, isFalse);  
});
```

`pumpEventQueue()` (từ package `test`) chờ các `Future` đang xếp hàng chạy xong — ở đây là lần tải đầu trong constructor.

Mock bằng mocktail — kiểm tra lời gọi

```
class MockReminderService extends Mock implements ReminderService {}

// ...
when(() => reminders.isSupported).thenReturn(true);
when(reminders.requestPermission).thenAnswer((_) async => true);
when(() => reminders.scheduleDaily(any())).thenAnswer((_) async {});
// ...
verify(reminders.requestPermission).called(1);
verify(() => reminders.scheduleDaily(const TimeOfDay(hour: 20, minute: 0))).called(1);
```

mocktail không cần sinh code (khác mockito ở cookbook). Dùng mock khi **lời gọi** là điều cần kiểm tra; còn lại dùng fake.

Widget test toàn app

```
Future<void> pumpVocab(WidgetTester tester, {String location = '/words', FakeTts? tts,
/* ... */}) async {
  await tester.pumpWidget(
    VocabApp(
      key: UniqueKey(),
      initialLocation: location,
      dependencies: fakeDependencies(tts: tts, /* ... */ httpClient: MockClient(()
async => http.Response(sampleJson, 200))),
    ),
  );
  await tester.pumpAndSettle();
}

testWidgets('danh sách → tìm không dấu (debounce) → phát âm', (tester) async {
  final tts = FakeTts();
  await pumpVocab(tester, tts: tts);
  expect(find.text('3 từ'), findsOneWidget);
  await tester.enterText(find.byType(TextField), 'hoa don');
  await tester.pump(const Duration(milliseconds: 300));
  await tester.pumpAndSettle();
  expect(find.text('1 từ'), findsOneWidget);
  await tester.tap(find.byTooltip('Phát âm invoice'));
  expect(tts.spoken, ['invoice']);
});
```

VocabApp nhận AppDependencies — main.dart truyền bản thật, test truyền bản giả. Không có dòng nào trong UI biết sự khác biệt.

Kết quả thật (2026-09-30, flutter test --reporter expanded test/app_test.dart):

```
00:00 +0: danh sách → tìm không dấu (debounce) → phát âm
00:01 +1: chi tiết: yêu thích, phát âm, tra từ điển online (MockClient)
00:02 +2: deep link /words/2 và id không tồn tại
00:02 +3: ôn tập: lật thẻ, "Đã nhớ" / "Chưa nhớ", hết lượt
00:02 +4: cài đặt: dark mode áp dụng cho cả app
00:03 +5: Lab mở được mọi chương
00:03 +6: All tests passed!
```

Và `test/ui/viewmodels_test.dart`:

```
00:00 +0: WordListViewModel tải lần đầu, tìm không dấu, lọc yêu thích
00:00 +1: WordListViewModel bỏ qua kết quả Cũ về muộn (giống switchMap)
00:00 +2: WordListViewModel lỗi → error, thử lại → hết lỗi
00:00 +3: ReviewViewModel (Command) lật thẻ, trả lời, hết lượt, thống kê
00:00 +4: SettingsViewModel theme đổi ngay và được lưu
00:00 +5: SettingsViewModel bật nhắc: xin quyền → đặt lịch (verify bằng mocktail)
00:00 +6: SettingsViewModel không có quyền / web → không đặt lịch, có thông báo
00:00 +7: SettingsViewModel tốc độ đọc được truyền cho TTS
00:00 +8: All tests passed!
```

(Đã bỏ các dòng `(setUpAll) / (tearDownAll) .`)

Integration test — app thật, dữ liệu thật

`examples/integration_test/app_test.dart`:

```
void main() {
  IntegrationTestWidgetsFlutterBinding.ensureInitialized();

  testWidgets('mở app, tìm "dam phan", mở chi tiết negotiate', (tester) async {
    await app.main();
    await tester.pumpAndSettle();
    expect(find.text('SỔ Từ Vựng'), findsOneWidget);
    await tester.enterText(find.byType(TextField), 'dam phan');
    await tester.pump(const Duration(milliseconds: 400));
    await tester.pumpAndSettle();
    await tester.tap(find.text('negotiate'));
    await tester.pumpAndSettle();
    expect(find.text('(v) đàm phán'), findsOneWidget);
  });
}
```

Chạy trên iPhone/Android (cắm máy): `flutter test integration_test/app_test.dart` — **NOT RUN** (không có thiết bị).

Chạy trên **Chrome headless** theo docs "Integration testing" (cần `chromedriver` cùng phiên bản với Chrome, file `test_driver/integration_test.dart`):

```
chromedriver --port=4444 &
flutter drive --driver=test_driver/integration_test.dart --
target=integration_test/app_test.dart \
-d web-server --browser-name=chrome --headless --no-web-resources-cdn
```

Sách gói lệnh này trong `scripts/integration-web.sh`. Kết quả thật (2026-09-30, Chromium 141 + chromedriver 141, log đầy đủ: [../logs/integration-web-vol2.txt](https://logs.integration-web-vol2.txt)), vài dòng cuối:

```
Launching integration_test/app_test.dart on Web Server in debug mode...
Waiting for connection from debug service on Web Server... 61.0s
All tests passed.
Application finished.
```

Test này chạy app **thật**: `main()` → SQLite Wasm → seed 24 từ → tìm "dam phan" → mở chi tiết.

Đi sâu

Chọn finder

`find.text`, `find.byType`, `find.byTooltip`, `find.byKey`, `find.bySemanticsLabel`, `find.widgetWithText(TextFormField, 'Email')`. Ưu tiên finder theo thứ **người dùng thấy** (chữ, tooltip, nhãn) — test vừa kiểm tra chức năng vừa kiểm tra trợ năng.

Thời gian giả

Trong `testWidgets`, `Timer` và `Future.delayed` chạy theo **đồng hồ giả**: phải `pump(duration)` để thời gian trôi. Debounce 300 ms → `pump(const Duration(milliseconds: 300))`. Trong `test()` thường (không phải `testWidgets`), thời gian là thật.

Ba bẫy sách đã gặp thật

1. **Pump lại app mà không đổi key** → State cũ (và router cũ) được giữ → test deep link thứ hai không đổi trang. Sửa: `key: UniqueKey()` (Tập 1, Chương 9).
2. **`pumpAndSettle` với animation lặp** (`PulsingDot`) → "pumpAndSettle timed out". Sửa: `pump(duration)` (Chương 5).
3. **Thiếu `setUpAll(() => registerFallbackValue(...))`** khi dùng `any()` của mocktail với kiểu tự định nghĩa → lỗi lúc chạy.

Coverage

`flutter test --coverage` tạo `coverage/lcov.info` (đã có trong `.gitignore`). Tập 3 (CI/CD) chạy trong pipeline.

Golden test

`matchesGoldenFile` so ảnh chụp widget với ảnh chuẩn — hữu ích cho UI ổn định, nhưng ảnh khác nhau giữa máy (font, nền tảng). Sách không dùng.

Lỗi và bẫy thường gặp

- **Gọi plugin thật trong widget test** → `MissingPluginException`. Tiêm fake.
- **Quên `await` trước `tester.tap/pump`** → kiểm tra quá sớm.
- **Test phụ thuộc thứ tự / dữ liệu dùng chung** → mỗi test tạo DB/fake mới (`setUp`).
- **Dùng mock cho mọi thứ** → test gắn chặt vào cách cài đặt; ưu tiên fake.
- **Integration test trên web cần chromedriver đúng phiên bản** với Chrome; lệch phiên bản → "session not created".

Tóm tắt

- Nhiều unit + widget test, ít integration test (theo docs).
- Fake cho dữ liệu/dịch vụ; mocktail khi cần `verify`.
- Widget test toàn app: tiêm `AppDependencies` giả.
- Integration test chạy app thật; trên web dùng `flutter drive` + `chromedriver` (đã chạy thật trong sandbox).

Bài tập (có lời giải)

Bài 1. Viết test chứng minh `WordListViewModel` hiện lỗi khi repository ném exception, và "Thử lại" xóa lỗi.

▼ Lời giải

```
test('lỗi → error, thử lại → hết lỗi', () async {
  final repo = FakeWordRepository()..failNextSearch = Exception('Ổ đĩa hỏng');
  final vm = WordListViewModel(repository: repo);
  await pumpEventQueue();
  expect(vm.error, isNotNull);
  await vm.refresh();
  expect(vm.error, isNull);
  expect(vm.words, hasLength(3));
});
```

Fake có "công tắc" `failNextSearch` — ném lỗi **một lần**. Trong app, `ErrorView` có nút "Thử lại" gọi `vm.refresh`.

Bài 2. Viết widget test cho luồng ôn tập: lật thẻ, trả lời "Đã nhớ" 1 lần và "Chưa nhớ" 2 lần, thấy "Bạn nhớ 1/3 từ", và repository ghi đủ 3 lần ôn.

▼ Lời giải

```
testWidgets('ôn tập: lật thẻ, "Đã nhớ" / "Chưa nhớ", hết lượt', (tester) async {
  final words = FakeWordRepository();
  await pumpVocab(tester, location: '/review', words: words);
  expect(find.text('Thẻ 1/3'), findsOneWidget);
  await tester.tap(find.text('Chạm để xem nghĩa'));
  await tester.pumpAndSettle(); // chờ animation lật xong
  expect(find.text('(n) ngân sách'), findsOneWidget);
  await tester.tap(find.text('Đã nhớ'));
  await tester.pumpAndSettle();
  for (var i = 0; i < 2; i++) {
    await tester.tap(find.text('Chạm để xem nghĩa'));
    await tester.pumpAndSettle();
    await tester.tap(find.text('Chưa nhớ'));
    await tester.pumpAndSettle();
  }
  expect(find.text('Bạn nhớ 1/3 từ'), findsOneWidget);
  expect(words.reviews, hasLength(3));
});
```

Test này nằm trong `test/app_test.dart` và pass (kết quả ở trên).

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-30 (đọc file nguồn docs ở commit `ab59c61`):

- Testing Flutter apps (overview) — <https://docs.flutter.dev/testing/overview> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/testing/overview.md>
- Cookbook: Introduction to widget testing — <https://docs.flutter.dev/cookbook/testing/widget/introduction> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/cookbook/testing/widget/introduction.md>
- Cookbook: Find widgets — <https://docs.flutter.dev/cookbook/testing/widget/finders> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/cookbook/testing/widget/finders.md>
- Cookbook: Introduction to unit testing — <https://docs.flutter.dev/cookbook/testing/unit/introduction> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/cookbook/testing/unit/introduction.md>
- Integration testing — <https://docs.flutter.dev/testing/integration-tests> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/testing/integration-tests/index.md>
- Case study — Testing each layer — <https://docs.flutter.dev/app-architecture/case-study/testing> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/testing/integration-tests/index.md>

38028f581a5/sites/docs/src/content/app-architecture/case-study/testing.md

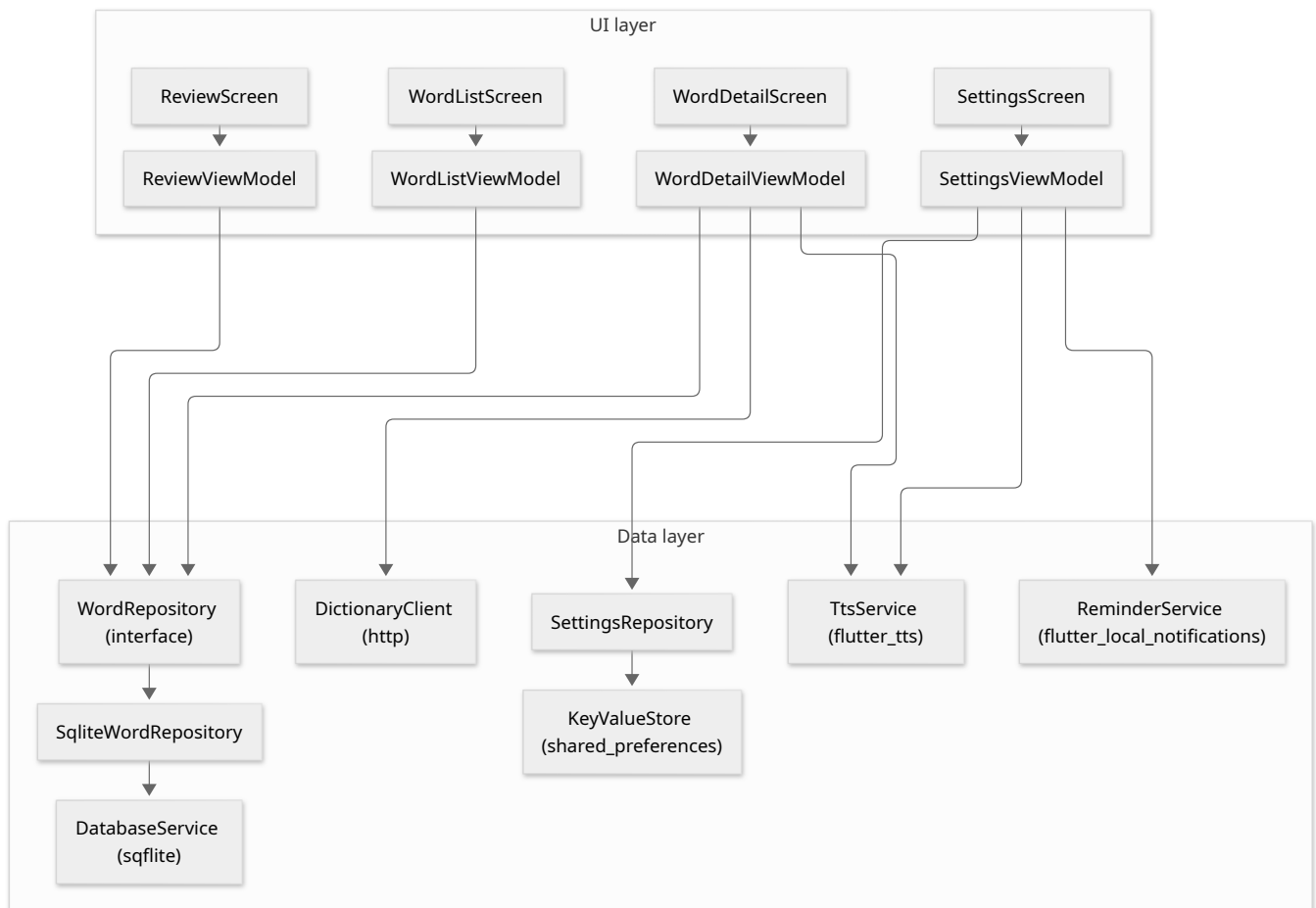
- Architecture recommendations (testing) — <https://docs.flutter.dev/app-architecture/recommendations> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/app-architecture/recommendations.md>
- Chrome for Testing (chromedriver 141.0.7390.37): <https://storage.googleapis.com/chrome-for-testing-public/141.0.7390.37/linux64/chromedriver-linux64.zip>
- pub.dev: <https://pub.dev/packages/mocktail>

Chương 8 — App mẫu Tập 2: "Sổ Từ Vựng" (bản tập dượt cho app TOEIC)

Mục tiêu

- Ghép mọi thứ của Tập 2 thành một app offline-first theo **kiến trúc chính thức**: data layer (service + repository), UI layer (View + ViewModel), DI bằng provider, điều hướng bằng go_router, Command + Result.
- Có đúng các tính năng app TOEIC (Task 9) cần: **SQLite offline**, **tìm có/không dấu**, **đọc to (TTS)**, **nhắc ôn hằng ngày (local notification)**, ôn tập bằng thẻ lật, **dark mode lưu bền**, test đầy đủ.
- Biết cấu trúc thư mục để Task 9 sao chép.

Giải thích đơn giản



Luồng dữ liệu **một chiều** (docs: "Use unidirectional data flow"): View gọi hàm của ViewModel → ViewModel gọi repository/service → cập nhật state → `notifyListeners()` → View build lại.

Ví dụ

Cấu trúc thư mục (theo gợi ý đặt tên của docs: `ui/core`, `data/repositories`, `data/services`)

```
examples/lib/
  main.dart          tạo DatabaseService + bản thật của mọi service →
runApp(VocabApp)
  app.dart           MultiProvider + MaterialApp.router (themeMode từ
SettingsViewModel)
  config/
    dependencies.dart AppDependencies: danh sách provider (thật hoặc giả)
    router.dart       go_router: /words, /words/:id, /review, /lab, /settings (4
tab)
  domain/
    models/word.dart  Word, ReviewStats (bất biến)
    word_of_the_day.dart hàm thuần (bài tập)
  data/
    services/         database_service, seed_words, key_value_store, tts_service,
                      reminder_service, dictionary_client, db_factory_stub/web
                      word_repository (interface), sqlite_word_repository,
    repositories/
settings_repository
  ui/
    core/             theme, home_shell, flip_card, error_view, debouncer
    word_list/ word_detail/ review/ settings/ mỗi thư mục: *_screen.dart +
*_viewmodel.dart
    utils/            result.dart, command.dart (mẫu chính thức, BSD),
vinamese.dart
  chapters/ch01..ch06/ ví dụ + lời giải từng chương
  lab/               tab Lab
examples/test/
  fakes/fakes.dart   FakeWordRepository, FakeTts, FakeReminderService
  data/              SQLite thật (FFI), settings, reminder, HTTP (MockClient)
  ui/viewmodels_test.dart ViewModel + mocktail
  app_test.dart      widget test toàn app
  chapters/          test từng chương
examples/integration_test/ app thật (chạy trên thiết bị hoặc Chrome)
examples/web/sqlite3.wasm, web/sqlite_sw.js  SQLite cho bản web
```

Điểm vào và DI

```
Future<void> main() async {  
  // Bắt buộc trước khi gọi plugin (sqlite, shared_preferences...) trước runApp.  
  WidgetsFlutterBinding.ensureInitialized();  
  configureDatabaseFactory();  
  final database = await DatabaseService.open();  
  runApp(  
    VocabApp(  
      dependencies: AppDependencies(  
        words: SqliteWordRepository(database),  
        settings: SettingsRepository(SharedPreferencesStore()),  
        tts: FlutterTtsService(),  
        reminders: LocalNotificationReminderService(),  
        dictionary: DictionaryClient(),  
      ),  
    ),  
  );  
}
```

```
List<SingleChildWidget> get providers => [  
  Provider<WordRepository>.value(value: words),  
  Provider<SettingsRepository>.value(value: settings),  
  Provider<TtsService>.value(value: tts),  
  Provider<ReminderService>.value(value: reminders),  
  Provider<DictionaryClient>.value(value: dictionary),  
  ChangeNotifierProvider<SettingsViewModel>(  
    create: (_) => SettingsViewModel(settings: settings, reminders: reminders, tts:  
tts)..load(),  
  ),  
];
```

Provider được khai báo theo **interface** (`WordRepository`, `TtsService`) để test thay bản giả.

Command trong ViewModel

`WordDetailViewModel` có 4 Command: `load`, `toggleFavorite`, `speak`, `lookup`. View lắng nghe chúng để khóa nút khi đang chạy và hiện lỗi:

```
OutlinedButton.icon(  
  onPressed: vm.toggleFavorite.running ? null : vm.toggleFavorite.execute,  
  icon: Icon(word.favorite ? Icons.star : Icons.star_border),  
  label: Text(word.favorite ? 'Bỏ yêu thích' : 'Yêu thích'),  
),
```

Chạy

```
cd books/flutter/vol2-trung-cap/examples
flutter pub get
flutter run -d chrome          # web (SQLite chạy bằng WebAssembly)
flutter run                    # iPhone qua Mac + Xcode
flutter test                   # 51 test
bash ../../scripts/integration-web.sh vol2-trung-cap # integration test trên Chrome
headless (cần chromedriver)
```

Nếu đổi phiên bản `sqlite_common_ffi_web`, chạy lại `dart run sqlite_common_ffi_web:setup` để tạo lại `web/sqlite3.wasm` + `web/sqlite_sw.js`, và ghim `sqlite3` cùng phiên bản với file `wasm`.

Kết quả kiểm tra thật (2026-09-30)

`bash books/flutter/scripts/check-all.sh vol2-trung-cap` ([../logs/check-all.txt](#)): format OK, `flutter analyze` → "No issues found!", `flutter test` → **51 test pass**, `flutter build web` → "✓ Built build/web", smoke test Chromium (390×844) thấy "Số Từ Vựng", "negotiate" → "WEB SMOKE OK". Integration test trên Chrome headless: "All tests passed." ([../logs/integration-web-vol2.txt](#)).

Chạy trên iPhone/Android: NOT RUN. Đọc to, thông báo theo lịch, SQLite native chỉ kiểm chứng được trên máy thật.

Đi sâu

Checklist sao chép sang app TOEIC (Task 9)

1. `pubspec.yaml`: dùng đúng phiên bản trong [STACK.md](#) mục 5 (kể cả `sqlite3: 3.6.0`).
2. Sao chép `utils/` (result, command, vietnamese), `ui/core/` (theme, flip_card, debouncer, error_view), `data/services/` (database, key_value_store, tts, reminder, db_factory_*), mẫu `config/dependencies.dart` + `router.dart`.
3. Mở rộng schema: bảng từ (lemma, word family, collocation...), bảng ôn tập theo SM-2 (app TOEIC đã có thuật toán SM-2 trong bản React Native, Task 5) — dùng migration `onUpgrade` như Chương 4.
4. Tìm kiếm có/không dấu: cột `search_key` + `withoutAccents`.
5. Cấu hình native: AppDelegate (thông báo), Gradle desugaring + receiver (Android), `INTERNET`.
6. Web: `web/sqlite3.wasm` + `web/sqlite_sw.js`; `build --no-web-resources-cdn`; không `--wasm` cho iPhone.
7. Test: fake cho mọi service; SQLite thật qua `sqlite_common_ffi`; widget test toàn app qua `AppDependencies`.

Vì sao ViewModel được tạo trong route?

`ChangeNotifierProvider(create: ...)` trong `builder` của `GoRoute`: ViewModel sống đúng bằng màn hình, tự `dispose`. `WordDetailScreen` thêm `key: ValueKey(id)` để mở từ khác thì tạo ViewModel mới.

Lỗi và bẫy thường gặp

- **Quên** `WidgetsFlutterBinding.ensureInitialized()` trước khi gọi plugin trong `main` → lỗi binding.
- **Khai báo provider theo lớp cụ thể** (`Provider<SqliteWordRepository>`) rồi đọc theo interface → `ProviderNotFoundException`.
- **Tạo router trong build** → mất trạng thái điều hướng khi đổi theme.
- **Import `sqlite_common_ffi_web` trực tiếp trong code dùng chung** → có thể hỏng build iOS/Android; dùng import có điều kiện.
- **Lệch phiên bản `sqlite3` và `sqlite3.wasm`** → lỗi mở DB trên web.

Tóm tắt

- App mẫu = kiến trúc chính thức thu nhỏ: services → repositories → ViewModels → Views; provider + go_router.
- Offline-first với SQLite; TTS và thông báo bọc sau interface; web chạy được nhờ SQLite Wasm.
- 51 test + integration test web, analyze 0 issue, build web OK — sẵn sàng làm khung cho app TOEIC.

Bài tập (có lời giải)

Bài 1. Thêm `resetProgress()` vào `WordRepository` để xóa toàn bộ lịch sử ôn (cho nút "Học lại từ đầu").

▼ Lời giải

Interface: `Future<int> resetProgress();`. Bản SQLite:

```
@override
Future<int> resetProgress() => _db.delete('reviews');
```

`delete` không có `where` xóa mọi dòng và trả về số dòng đã xóa. Bản fake xóa list `reviews` và đặt lại `reviewCount`. Test (`database_test.dart`, "Bài 1 (Ch.8)": ôn 2 lần → `resetProgress()` trả 2 → `reviewCount` về 0. Thêm hàm vào interface buộc **mọi** cài đặt (thật và giả) phải cập nhật — compiler nhắc nếu quên.

Bài 2. Viết `wordOfTheDay(words, day)`: trong cùng một ngày luôn ra cùng một từ, sang ngày khác đổi từ, không phụ thuộc thứ tự danh sách đầu vào.

▼ Lời giải

`examples/lib/domain/word_of_the_day.dart`:

```
Word? wordOfTheDay(List<Word> words, DateTime day) {
  if (words.isEmpty) return null;
  final sorted = [...words]..sort((a, b) => a.id.compareTo(b.id));
  final dayNumber = DateTime.utc(day.year, day.month,
    day.day).millisecondsSinceEpoch ~/ Duration.millisecondsPerDay;
  return sorted[dayNumber % sorted.length];
}
```

Không dùng `Random` → test được. Kết quả thật (`test/domain_test.dart`):

```
00:00 +0: Word: accuracy, copyWith, ==
00:00 +1: Bài 2 (Ch.8): wordOfTheDay ổn định trong ngày, đổi theo ngày
00:00 +2: All tests passed!
```

Nối vào UI: trong `ReviewViewModel._start`, gọi `wordOfTheDay(await _repository.search(), _clock())` và hiện ở đầu màn hình Ôn tập.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-30 (đọc file nguồn docs ở commit `ab59c61`):

- Guide to app architecture — <https://docs.flutter.dev/app-architecture/guide> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/app-architecture/guide.md>
- Architecture recommendations — <https://docs.flutter.dev/app-architecture/recommendations> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/app-architecture/recommendations.md>
- Case study — Dependency injection — <https://docs.flutter.dev/app-architecture/case-study/dependency-injection> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/app-architecture/case-study/dependency-injection.md>
- Design pattern: Command — <https://docs.flutter.dev/app-architecture/design-patterns/command> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/app-architecture/design-patterns/command.md>

- Mã nguồn mẫu Command (BSD): <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/examples/app-architecture/command/lib/command.dart>
- Design pattern: Offline-first — <https://docs.flutter.dev/app-architecture/design-patterns/offline-first> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/app-architecture/design-patterns/offline-first.md>
- App TOEIC (React Native, Task 5) trong repo: [apps/toEIC/README.md](#)

GLOSSARY — Thuật ngữ Flutter (Anh → Việt)

Thuật ngữ (English)	Giải thích tiếng Việt	Gần giống trong Angular / React Native
Flutter	Bộ công cụ UI của Google: một codebase Dart → app iOS, Android, web, desktop; tự vẽ mọi pixel	Angular (web) / React Native (mobile)
Dart	Ngôn ngữ lập trình của Flutter: có kiểu, null safety, biên dịch AOT/JIT/JS/Wasm	TypeScript
Widget	Mô tả bất biến (immutable) của một phần giao diện; "mọi thứ là widget"	Component
Widget tree (cây widget)	Các widget lồng nhau tạo thành cây	Cây component / DOM
StatelessWidget	Widget không có state riêng; chỉ phụ thuộc tham số	Component "dumb" / presentational
StatefulWidget	Widget có object <code>State</code> sống lâu, đổi bằng <code>setState</code>	Component có state
State (object)	Nơi giữ dữ liệu thay đổi và các hàm vòng đời <code>initState</code> , <code>dispose</code>	Class component + lifecycle hooks
<code>build()</code>	Hàm trả về cây widget ứng với state hiện tại	Template / render
<code>setState()</code>	Báo Flutter state đã đổi → build lại widget	<code>signal.set()</code> / <code>setState</code> của React
BuildContext	"Vị trí" của widget trong cây; dùng để tìm Theme, Navigator, provider ở trên	Injector hiện tại
Element	Đối tượng Flutter tạo từ widget để quản lý cây thật (ít khi dùng trực tiếp)	—
RenderObject	Đối tượng làm layout và vẽ (tầng thấp)	—
Key	Định danh giúp Flutter giữ đúng state khi danh sách đổi thứ tự	<code>track</code> trong <code>@for</code> / <code>key</code> của React
Constraints (ràng buộc)	Cha nói cho con kích thước tối thiểu/tối đa; con chọn kích thước trong đó	— (khác CSS)
Row / Column / Flex	Bố cục ngang / dọc	Flexbox <code>flex-direction</code>
Expanded / Flexible	Con chiếm phần còn lại của Row/Column	<code>flex: 1</code>
Material 3	Hệ thiết kế của Google, mặc định trong Flutter	Angular Material
Cupertino	Bộ widget kiểu iOS	—
ThemeData / ColorScheme	Cấu hình màu, chữ toàn app; hỗ trợ sáng/tối	Theme SCSS / CSS variables
Sliver	Mảnh của vùng cuộn (SliverList, SliverAppBar) cho hiệu ứng cuộn tùy biến	Virtual scroll (CDK)
Navigator	Ngăn xếp (stack) màn hình kiểu mệnh lệnh: <code>push</code> / <code>pop</code>	<code>router.navigate</code> + history
go_router	Package định tuyến theo URL (declarative), docs chính thức khuyên dùng	Angular Router
Deep link	Mở app thẳng vào một màn hình bằng URL	Route URL
Redirect (go_router)	Hàm chuyển hướng trước khi hiển thị route	Route guard <code>canActivate</code>

Thuật ngữ (English)	Giải thích tiếng Việt	Gần giống trong Angular / React Native
ShellRoute / StatefulShellRoute	Route "khung" bọc các route con (tab bar)	Layout component + <code><router-outlet></code>
Future	Một giá trị sẽ có trong tương lai	Promise
Stream	Chuỗi nhiều giá trị theo thời gian	Observable (RxJS)
<code>async</code> / <code>await</code>	Viết code bất đồng bộ như đồng bộ	Giống TypeScript
Isolate	"Luồng" riêng có bộ nhớ riêng để chạy việc nặng	Web Worker
ChangeNotifier	Class có sẵn, gọi <code>notifyListeners()</code> để báo đổi	Service có state + Subject/Signal
ValueNotifier	ChangeNotifier giữ một giá trị	<code>signal()</code>
ListenableBuilder / ValueListenableBuilder	Widget build lại khi Listenable báo đổi	<code>async pipe</code> / đọc signal trong template
InheritedWidget	Cơ chế truyền dữ liệu xuống cây, con tìm lên bằng context	Hierarchical injector
provider	Package DI + lắng nghe ChangeNotifier, docs chính thức khuyên dùng	Angular DI (<code>providers</code> , <code>inject</code>)
MVVM	Model–View–ViewModel: View = widget, ViewModel = ChangeNotifier	Component + service/facade
Repository	Lớp che giấu nguồn dữ liệu (SQLite, API) khỏi UI	Data service
Command (pattern)	Đối tượng bọc một hành động bất đồng bộ + trạng thái chạy/lỗi	NgRx effect đơn giản
Hot reload	Nạp lại code khi đang chạy, giữ state (phím <code>r</code>)	HMR / Fast Refresh
Hot restart	Khởi động lại app, mất state (phím <code>R</code>)	Tải lại trang
pub / pub.dev	Trình quản lý package và kho package của Dart	npm / npmjs.com
<code>pubspec.yaml</code> / <code>pubspec.lock</code>	Khai báo phụ thuộc / khóa phiên bản	<code>package.json</code> / <code>package-lock.json</code>
Plugin	Package có code native (Swift/Kotlin)	Native module (RN)
Platform channel (MethodChannel)	Kênh gọi qua lại giữa Dart và code native	TurboModule / bridge (RN)
Pigeon	Công cụ sinh code type-safe cho platform channel	Codegen (RN)
Swift Package Manager (SwiftPM)	Trình quản lý phụ thuộc iOS, mặc định từ Flutter 3.44	CocoaPods
AOT / JIT	Biên dịch trước (bản release) / biên dịch lúc chạy (debug, hot reload)	<code>ng build</code> / <code>ng serve</code>
Debug / Profile / Release mode	3 chế độ build: gỡ lỗi / đo hiệu năng / phát hành	dev / prod build
DevTools	Bộ công cụ gỡ lỗi, đo hiệu năng của Flutter	Chrome DevTools, Angular DevTools
Impeller	Engine vẽ mới của Flutter (mặc định trên iOS)	—

Thuật ngữ (English)	Giải thích tiếng Việt	Gần giống trong Angular / React Native
CanvasKit / skwasm	Bộ vẽ của Flutter web (Skia biên dịch sang WebAssembly)	—
Semantics	Cây thông tin trợ năng (VoiceOver đọc); test cũng tìm theo semantics	ARIA
Widget test	Test một widget trong môi trường giả, không cần thiết bị	Component test (TestBed)
Integration test	Test cả app trên thiết bị/giả lập (package integration_test)	E2E (Playwright/Cypress)
Fake / Mock	Bản giả tự viết (fake) / đối tượng giả kiểm tra lời gọi (mock, mocktail)	<code>jasmine.createSpyObj</code>
sqflite	Plugin SQLite cho iOS/Android	expo-sqlite
shared_preferences	Lưu cặp key-value nhỏ	localStorage / AsyncStorage
TTS (text-to-speech)	Đọc chữ thành giọng nói (flutter_tts)	expo-speech
Local notification	Thông báo do app tự đặt lịch, không cần server	expo-notifications (local)
Keychain / Keystore	Kho bí mật của iOS / Android (flutter_secure_storage)	expo-secure-store
Obfuscation	Làm rối tên hàm/lớp trong bản build để khó dịch ngược	Minify
Flavor	Biến thể build (dev/staging/prod)	Angular environments
TestFlight	Kênh phát bản thử iOS của Apple	—